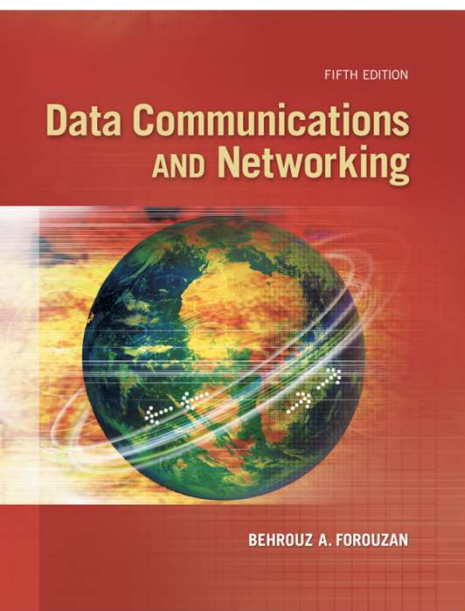


Computer Networks

Unicast Routing

Dr.N.Sujaudeen

Assistant Professor / CSE



AGENDA

Routing Algorithms

- Distance Vector

- Link state Routing

- Path Vector Routing

Routing Protocols

- Routing Information Protocol RIP

- Open Shortest Path First OSPF

- Border Gateway Protocol BGP



ROUTING

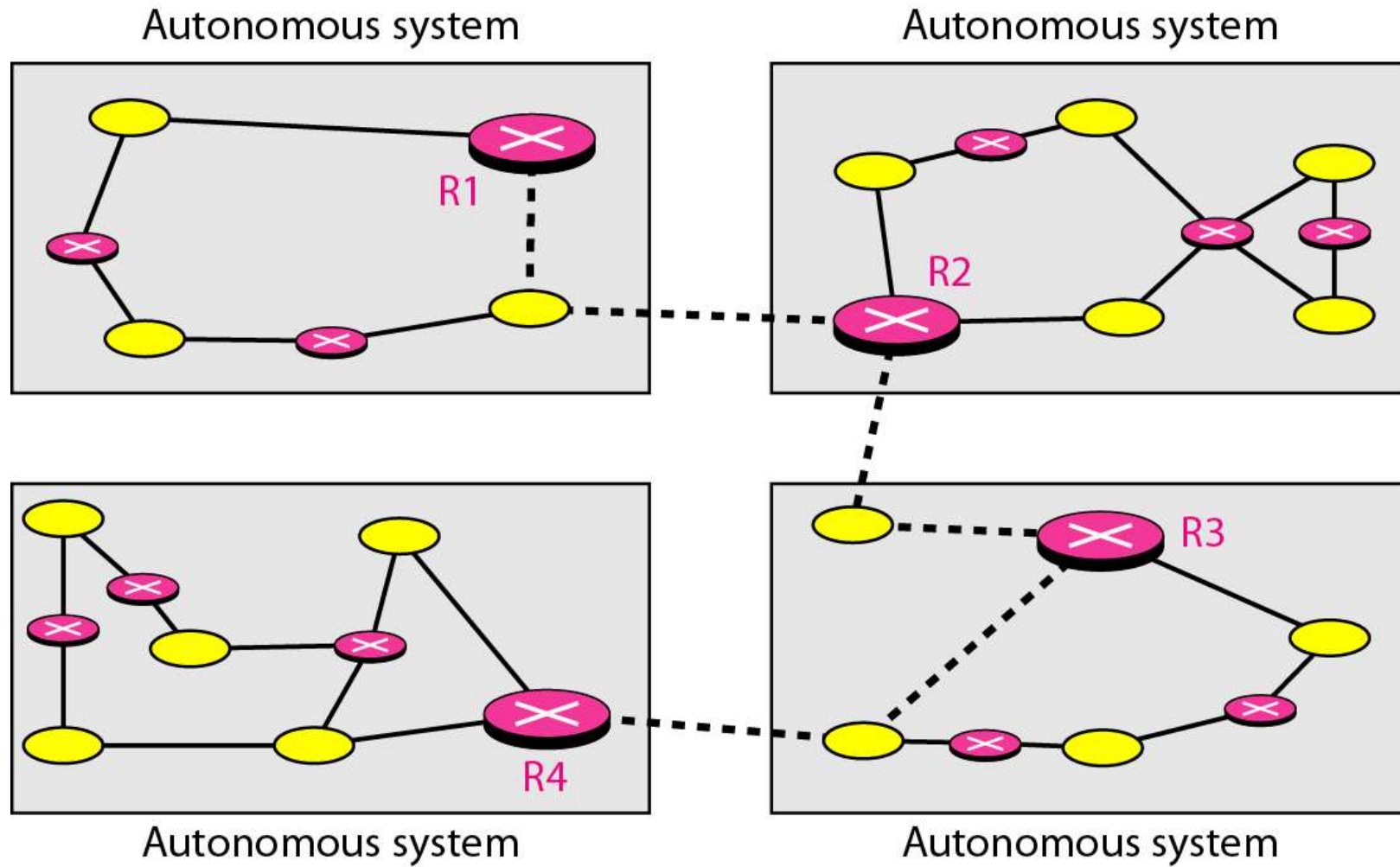
- *A routing table can be either static or dynamic. A static table is one with manual entries.*
- *A dynamic table is one that is updated automatically when there is a change somewhere in the Internet.*
- *A routing protocol is a combination of rules and procedures that lets routers in the Internet inform each other of changes.*



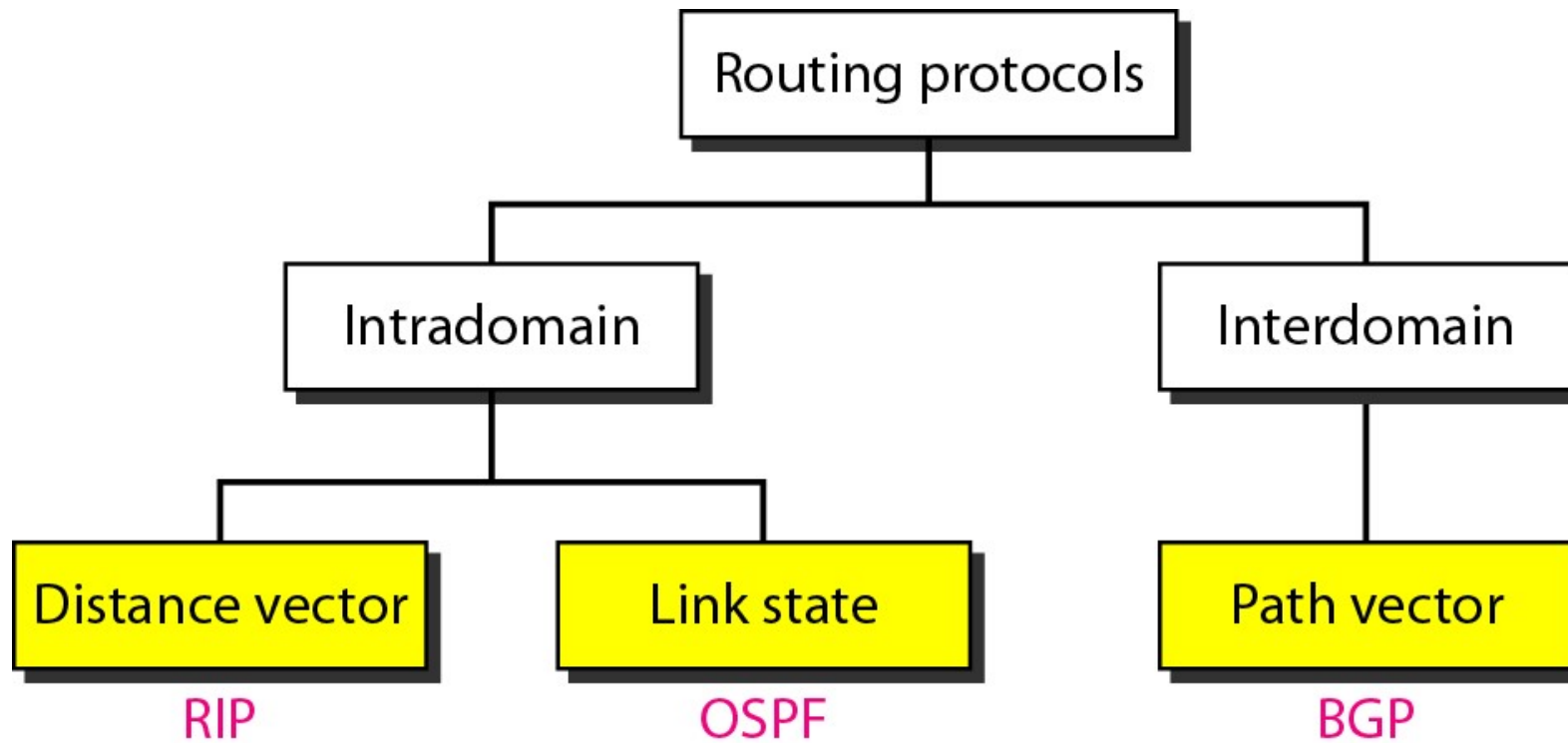
General Idea

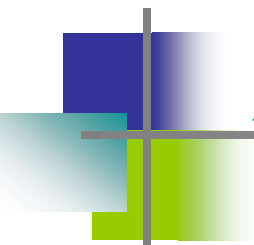
- In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables.
- The source host needs no forwarding table because it delivers its packet to the default router in its local network.
- The destination host needs no forwarding table either because it receives the packet from its default router in its local network.
- This means that only the routers that glue together the networks in the internet need forwarding tables.

Autonomous systems



Popular routing protocols





Least-Cost Routing

When an internet is modeled as a weighted graph, one of the ways to interpret the best route from the source router to the destination router is to find the least cost between the two.

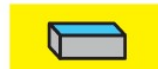
Least cost among all possible routes.

An internet and its graphical representation

Legend



Router



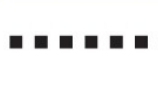
LAN



Edge

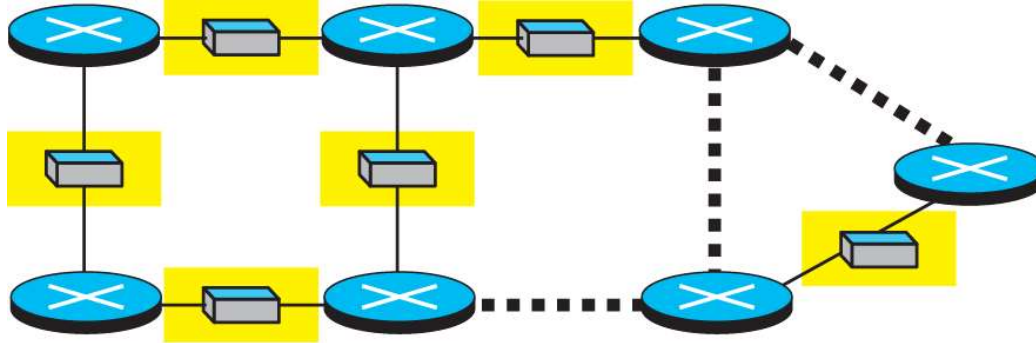


Node

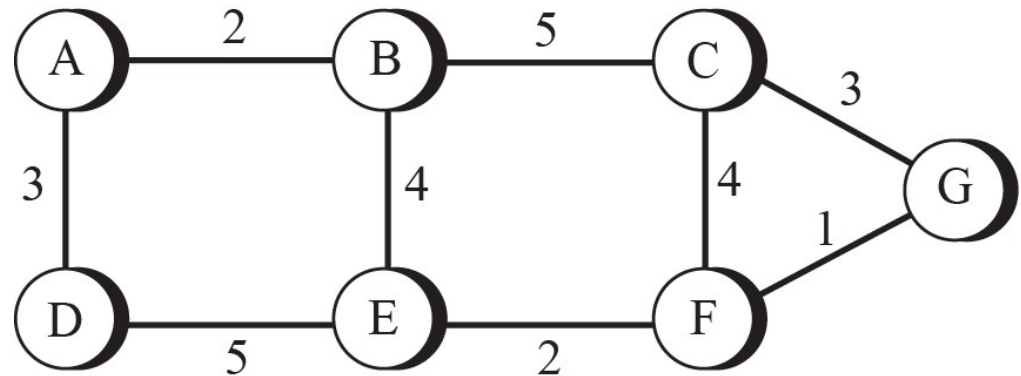


WAN

2, 3, ... Costs

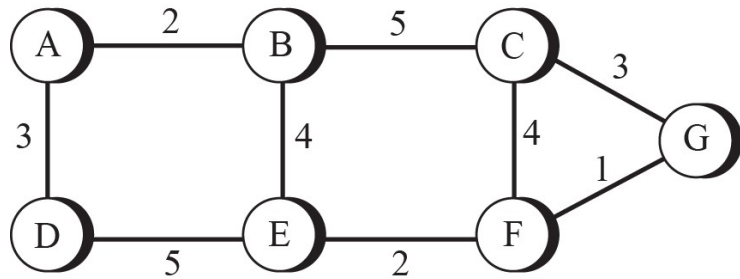
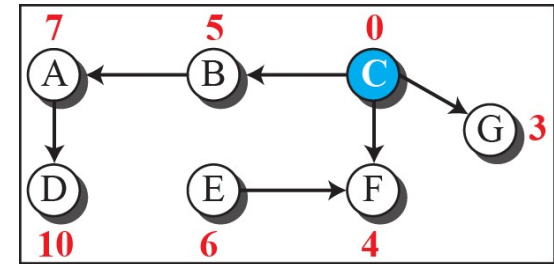
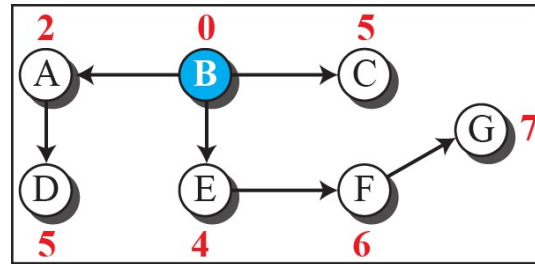
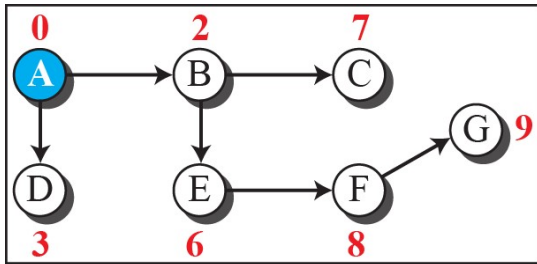


a. An internet



b. The weighted graph

Least-cost trees for nodes in the internet



b. The weighted graph

Legend

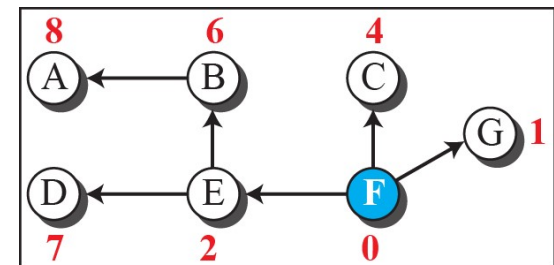
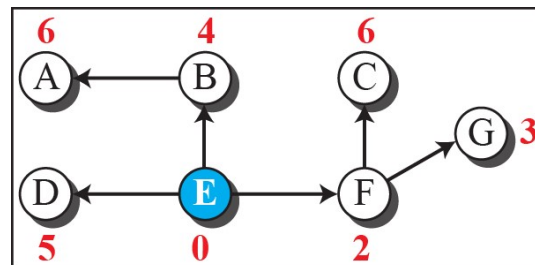
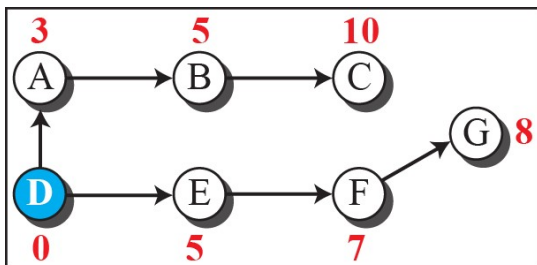
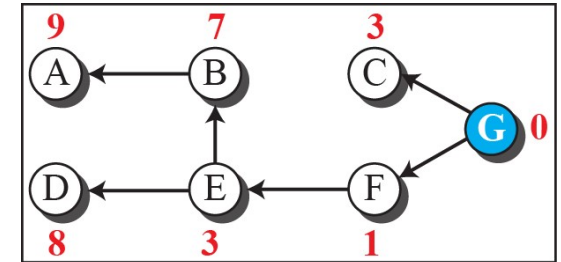


Root of the tree



Intermediate or end node

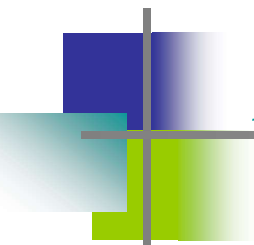
1, 2, ... Total cost from the root



ROUTING ALGORITHMS

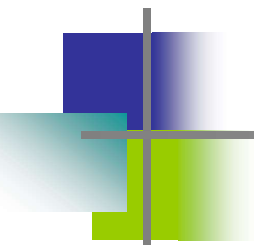
Several routing algorithms have been designed in the past.

The differences between these methods are in the way they interpret the least cost and the way they create the least-cost tree for each node.



Distance-Vector Routing

- The distance-vector (DV) routing - find the best route.
- Each node creates its own least-cost tree with the rudimentary information it has about its immediate neighbors.
- The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet.
- Router continuously tells all of its neighbors what it knows about the whole internet (although the knowledge can be incomplete).



Note

In distance vector routing, each node shares its routing table(whole internet) with its immediate neighbors periodically and when there is a change.

Bellman Ford Algorithm –Least Cost

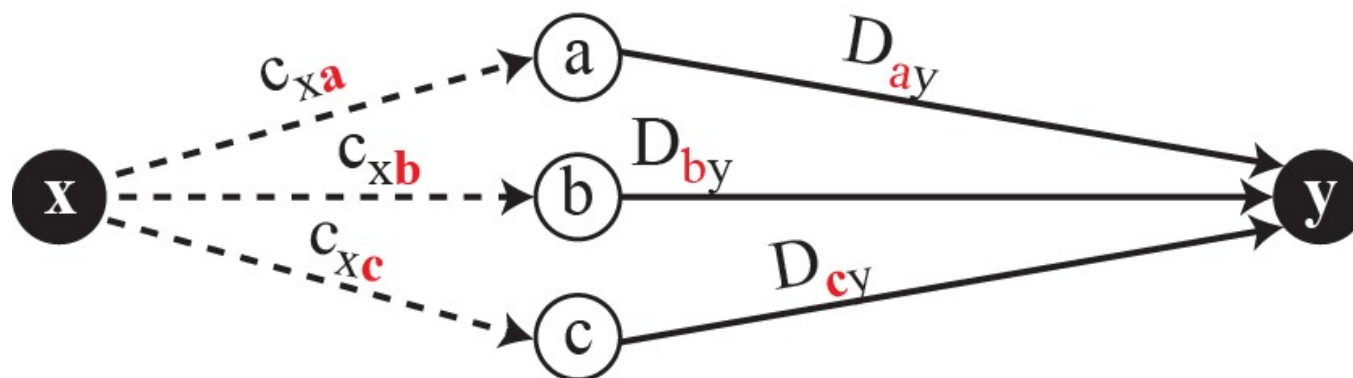
To find the least cost (shortest distance) between a source node, x , and a destination node, y , through some intermediary nodes ($a, b, c, \dots$) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and the destination are given.

The following shows the general case in which D_{xy} is the shortest distance and c_{ij} is the cost between nodes i and j .

$$D_{xy} = \min \{ (c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \dots \}$$

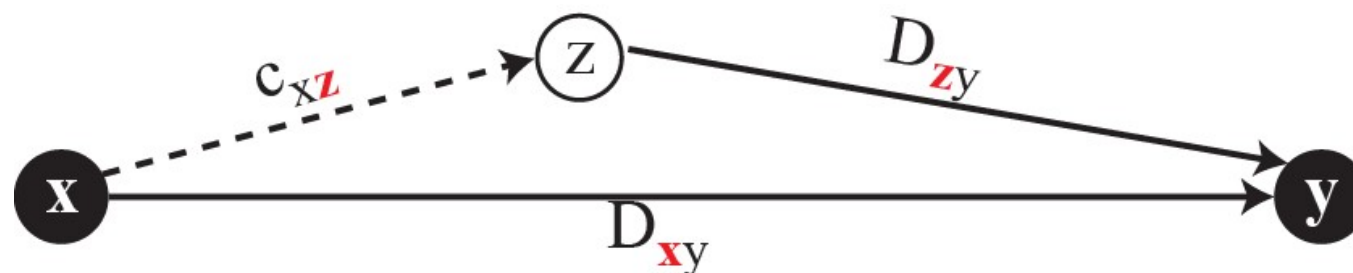
$$D_{xy} = \min \{ D_{xy}, (c_{xz} + D_{zy}) \}$$

Graphical idea behind Bellman-Ford equation



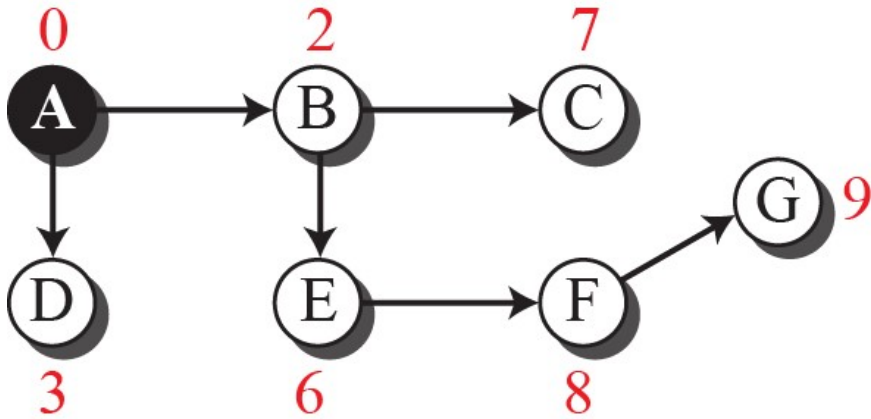
a. General case with three intermediate nodes

$$D_{xy} = \min \{ D_{xy}, (c_{xz} + D_{zy}) \}$$



b. Updating a path with a new route

The distance vector corresponding to a tree

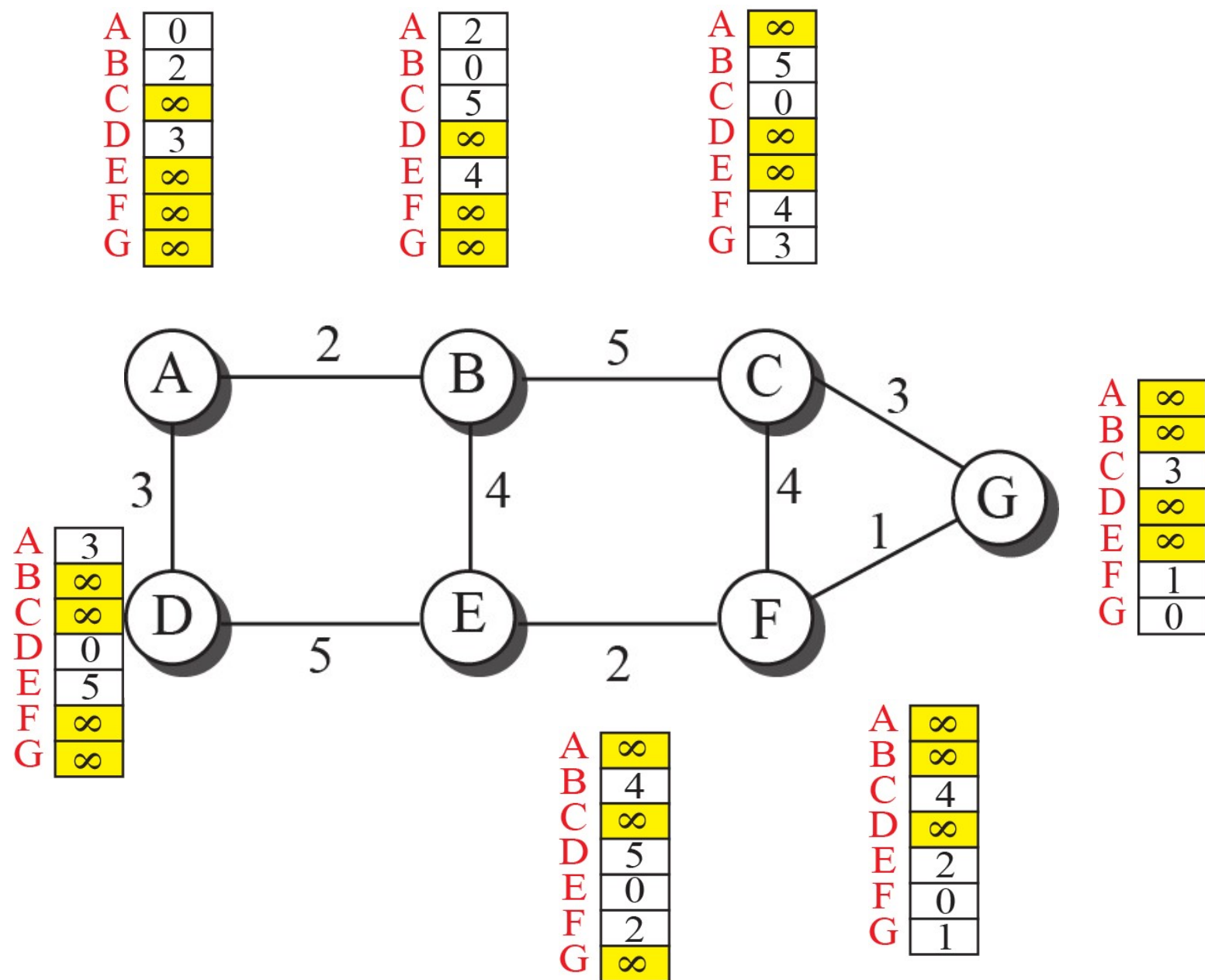


a. Tree for node A

	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

The first distance vector for an internet



Updating distance vectors

New B		Old B		A	
A	2	A	2	A	0
B	0	B	0	B	2
C	5	C	5	C	∞
D	5	D	∞	D	3
E	4	E	4	E	∞
F	∞	F	∞	F	∞
G	∞	G	∞	G	∞

$B[\] = \min (B[\], 2 + A[\])$

Note:

$X[\]$: the whole vector

a. First event: B receives a copy of A's vector.

New B		Old B		E	
A	2	A	2	A	∞
B	0	B	0	B	4
C	5	C	5	C	∞
D	5	D	5	D	5
E	4	E	4	E	0
F	6	F	∞	F	2
G	∞	G	∞	G	∞

$B[\] = \min (B[\], 4 + E[\])$

b. Second event: B receives a copy of E's vector.

Distance-Vector Routing Algorithm for A Node

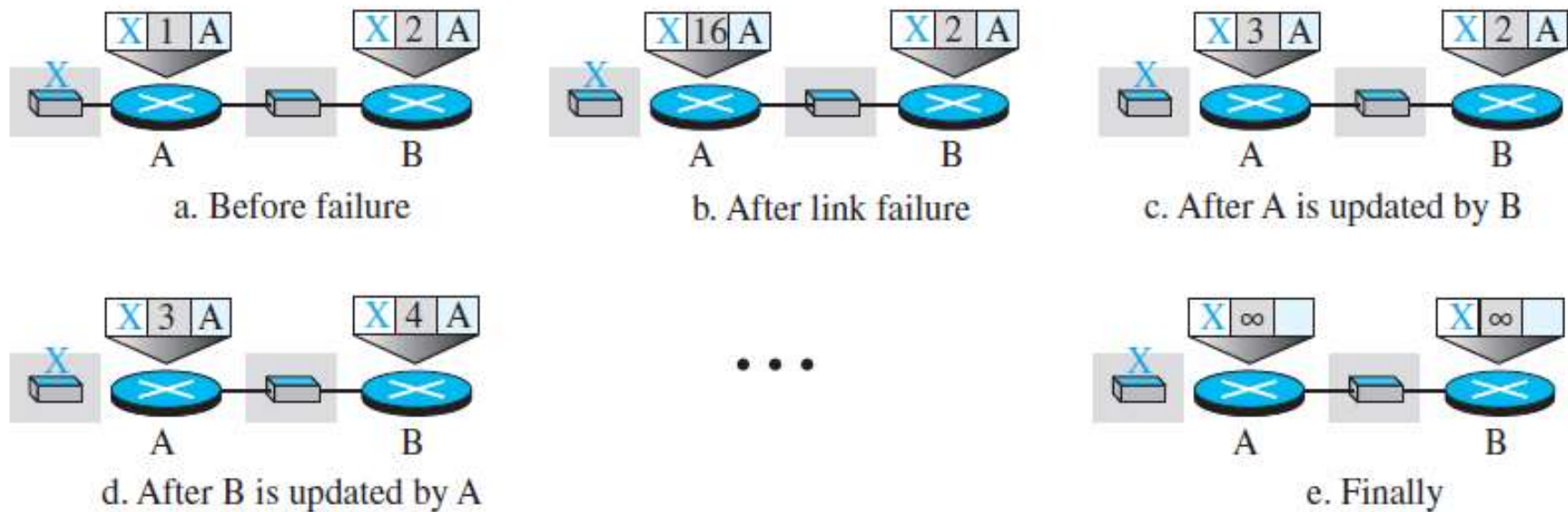
```
1 Distance_Vector_Routing ( )
2 {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7         if (y is a neighbor)
8             D[y] = c[myself][y]
9         else
10            D[y] =  $\infty$ 
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16        wait (for a vector  $D_w$  from a neighbor w or any change in the link)
17        for (y = 1 to N)
18        {
19            D[y] = min [D[y], (c[myself][w] +  $D_w[y]$ )]           // Bellman-Ford equation
20        }
21        if (any change in the vector)
22            send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 } // End of Distance Vector
```

When to share

- Periodic update – every 30s
- Triggered update
 1. A node receives a table from a neighbour, resulting in changes in its own table after updating.
 2. A node detects some failure in the neighbouring links which results in a distance change to infinity.

Two Node Instability (Count to infinity problem)

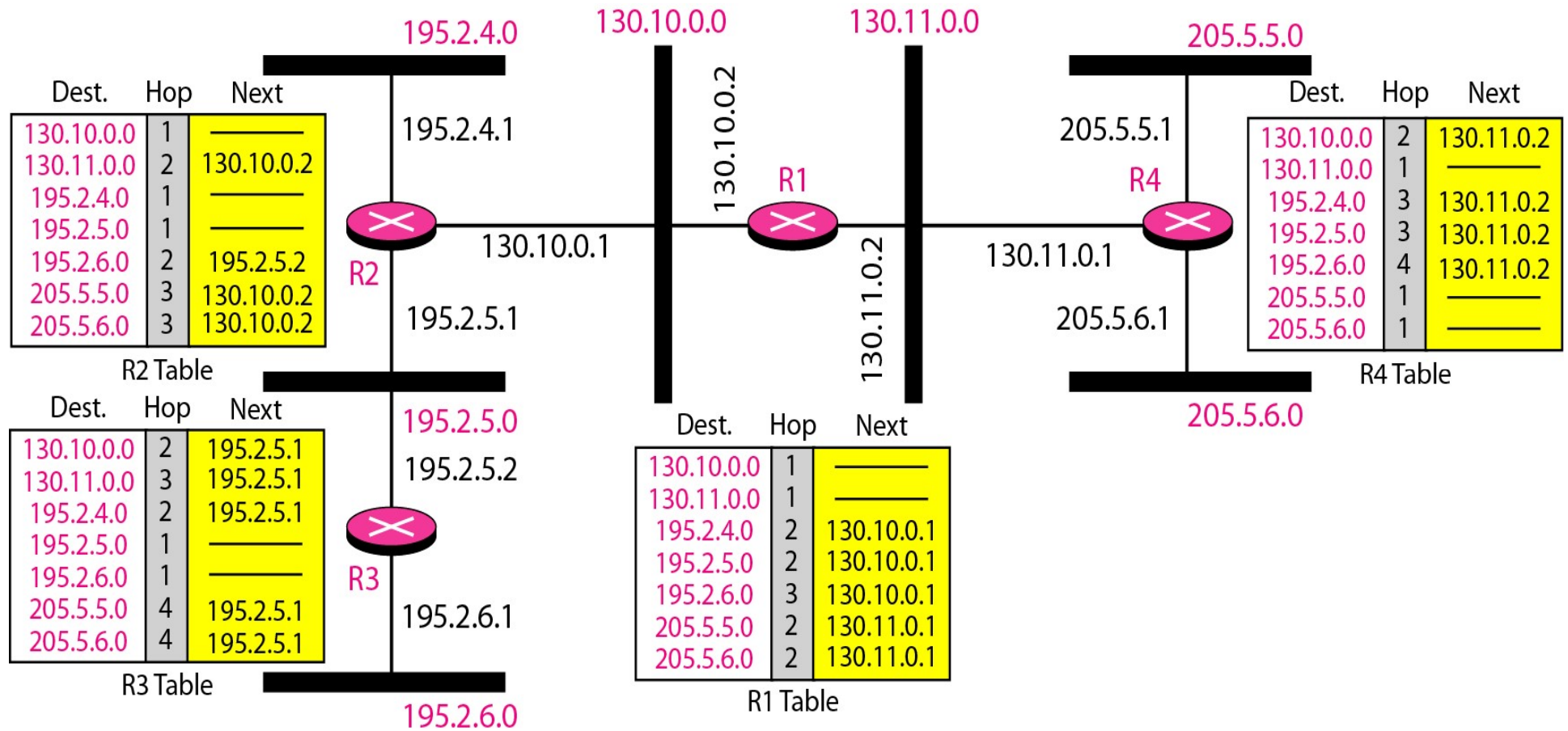
Figure 20.7 Two-node instability

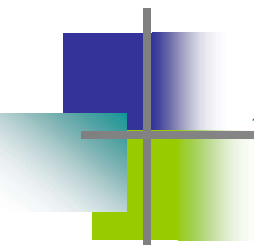


Solution

- Split horizon – don't send routes learnt from neighbour back to it
- Split horizon with poison reverse –
 - corresponding protocol uses a timer, and if there is no news about a route, the node deletes the route from its table.
 - send routes learnt from neighbour back to it with infinity value (it is better to get bad news than no news)

Example of a domain using RIP



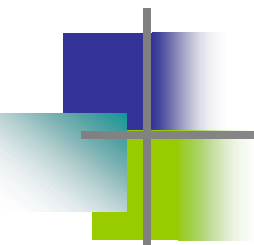


Link-State Routing

Link-state to define the characteristic of a link (an edge) that represents a network in the internet.

Cost associated with an edge defines the state of the link.

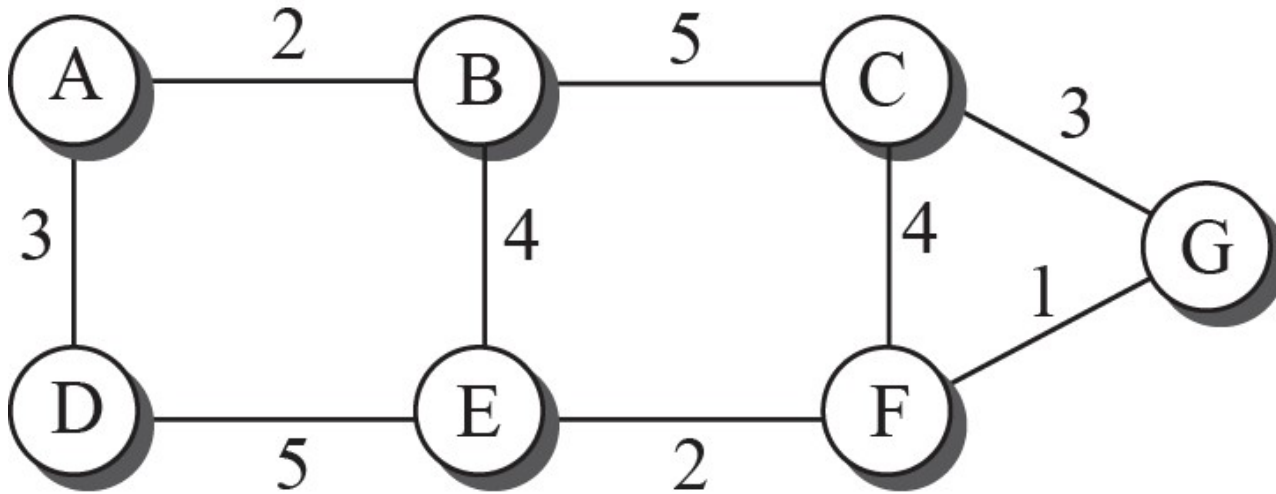
Links with lower costs are preferred to links with higher costs; if the cost of a link is infinity, it means that the link does not exist or has been broken.



Note

In link state routing, each node shares(floods) about its immediate neighbors information to the whole network.

Example of a link-state database (LSDB)



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

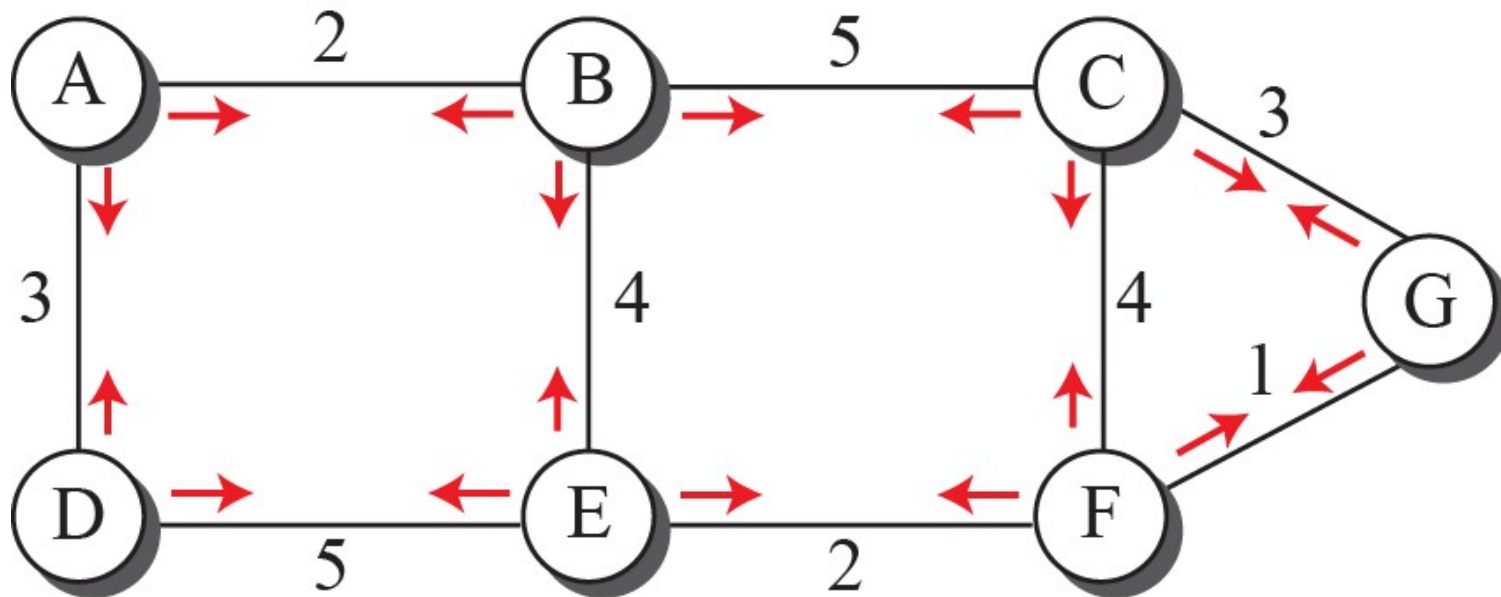
b. Link state database

LSPs created and sent out by each node to build LSDB

Node	Cost
B	2
D	3

Node	Cost
A	2
C	5
E	4

Node	Cost
B	5
F	4
G	3



Node	Cost
C	3
F	1

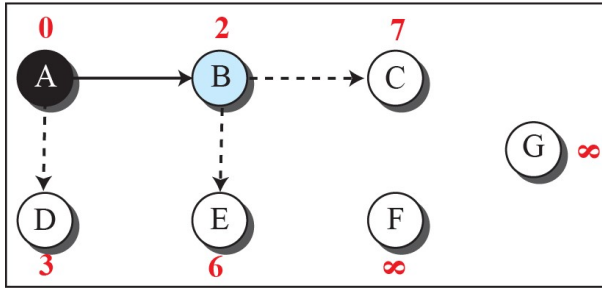
Node	Cost
A	3
E	5

Node	Cost
B	4
D	5
E	2

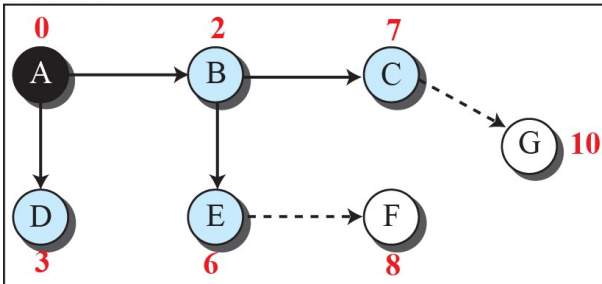
Node	Cost
C	4
E	2
G	1

Least-cost tree

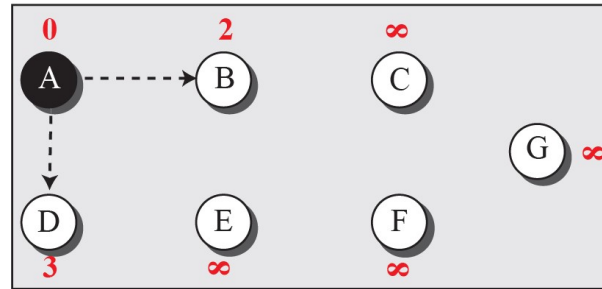
Iteration 1



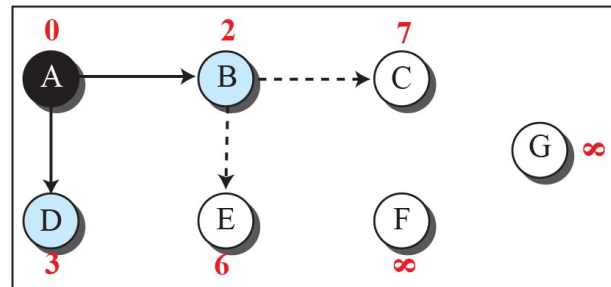
Iteration 4



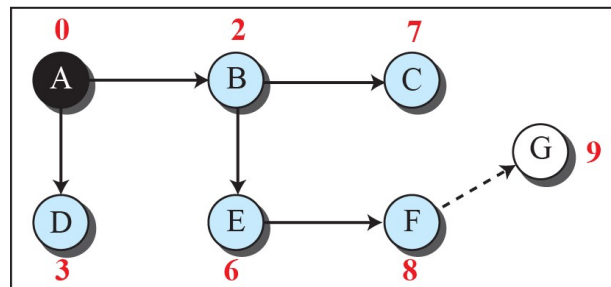
Initialization



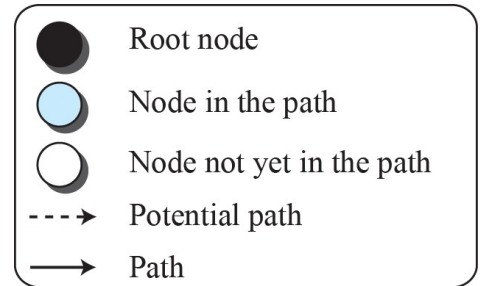
Iteration 2



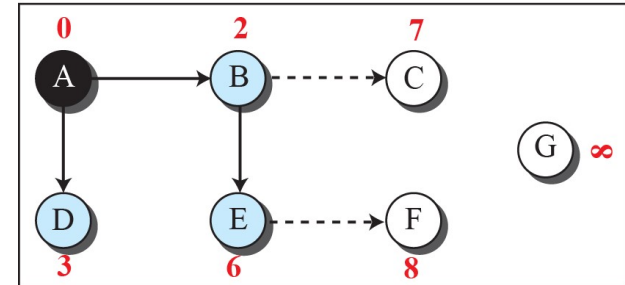
Iteration 5



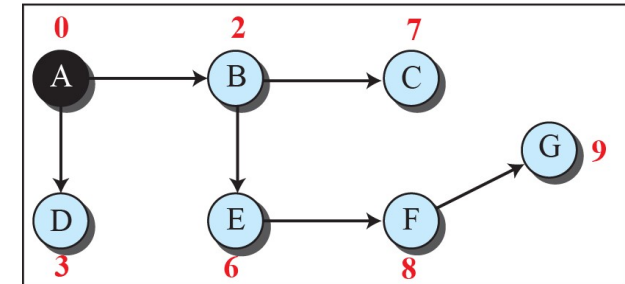
Legend



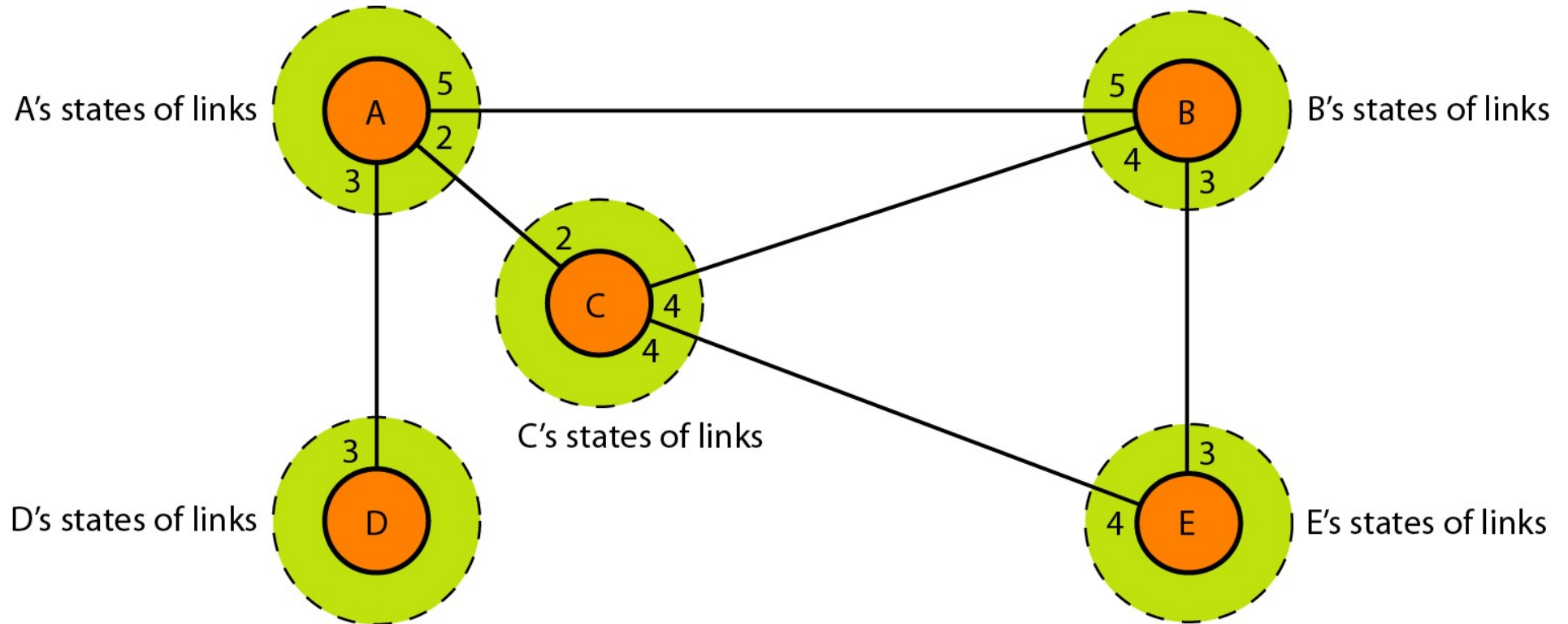
Iteration 3



Iteration 6



Link State Knowledge –Another Example



Link State Knowledge

Building Routing Tables

In link state routing, four sets of actions are required to ensure that each node has the routing table showing the least-cost node to every other node.

1. Creation of the states of the links by each node, called the link state packet (LSP).
2. Dissemination of LSPs to every other router, called **flooding**, in an efficient and reliable way.
3. Formation of a shortest path tree for each node.
4. Calculation of a routing table based on the shortest path tree.

When to share

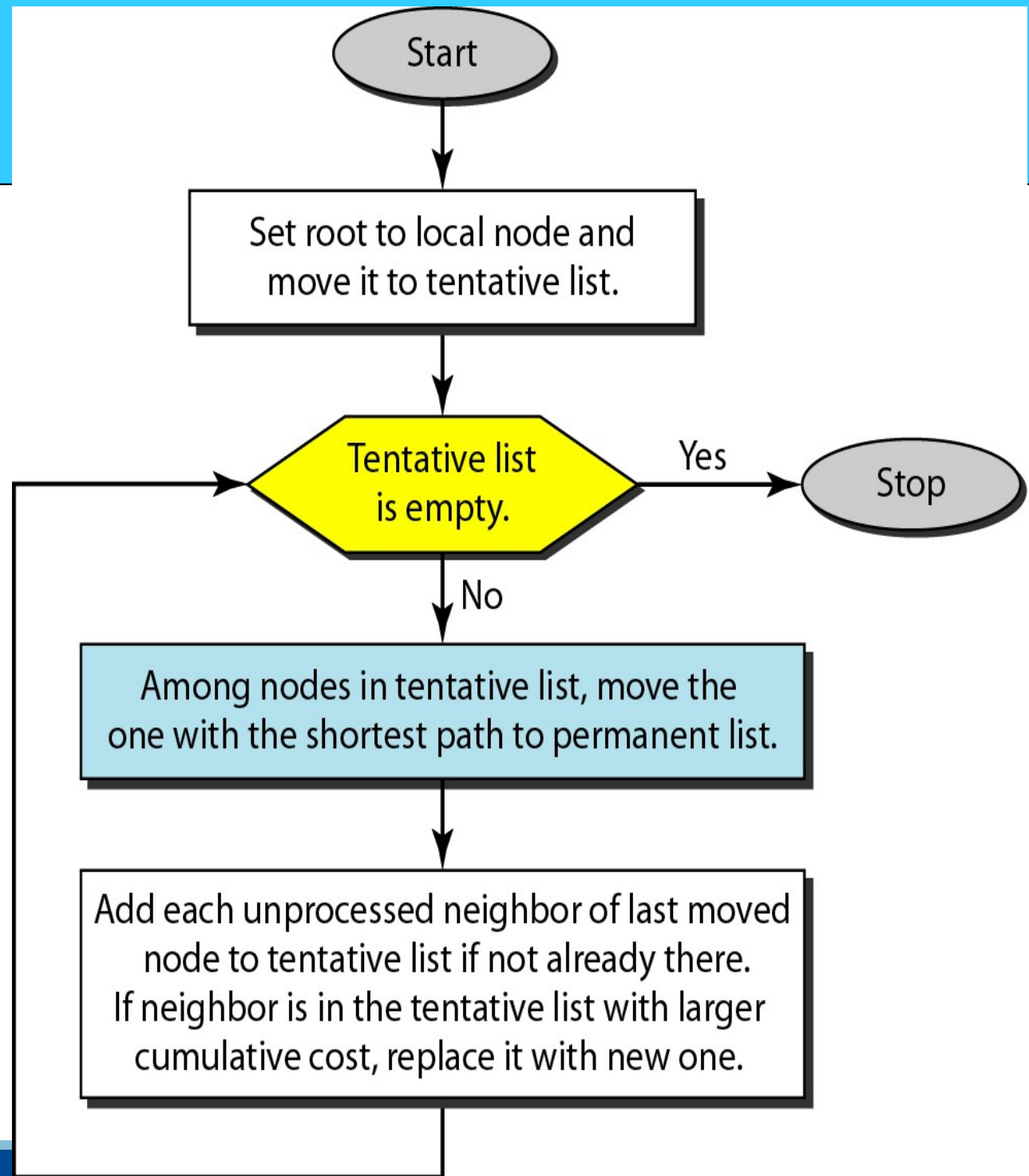
- Periodic update – every 60m to 2 hr
- Change in topology

Flooding of LSP

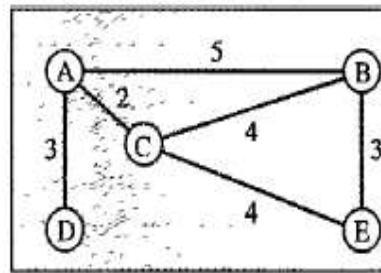
Flooding of LSPs After a node has prepared an LSP, it must be disseminated to all other nodes, not only to its neighbors. The process is called flooding and based on the following:

1. The creating node sends a copy of the LSP out of each interface.
2. A node that receives an LSP compares it with the copy it may already have. If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP. If it is newer, the node does the following:
 - a. It discards the old LSP and keeps the new one.
 - b. It sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the domain (where a node has only one interface).

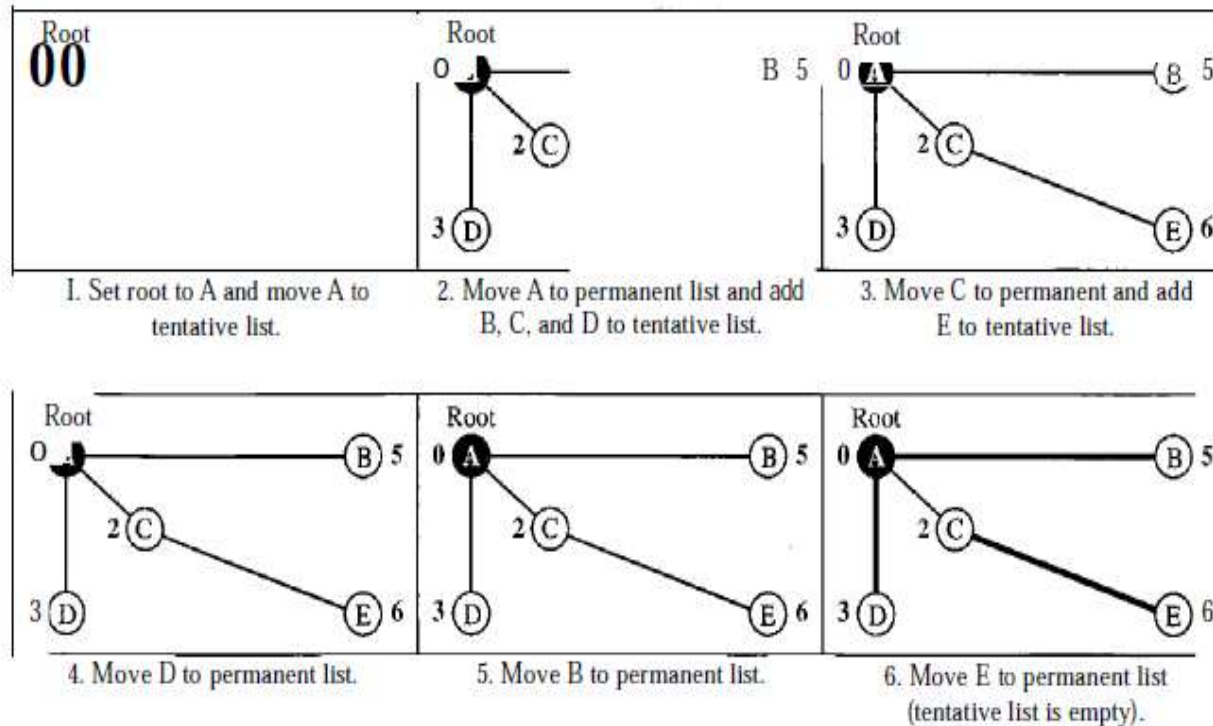
Dijkstra Algo



Shortest path



Topology



Permanent List	Tentative list
Empty	A(0)
A(0)	B(5)C(2)D(3)
A(0) C(2)	B(5)D(3) E(6)
A(0) C(2) D(3)	B(5) E(6)
A(0) C(2) D(3) B(5)	E(6)
A(0) C(2) D(3) B(5)E(6)	

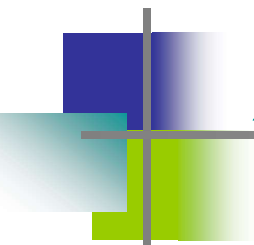
Routing table

Permanent List	Tentative list
Empty	A(0)
A(0)	B(5)C(2)D(3)
A(0) C(2)	B(5)D(3) E(6)
A(0) C(2) D(3)	B(5) E(6)
A(0) C(2) D(3) B(5)	E(6)
A(0) C(2) D(3) B(5)E(6)	

<i>Node</i>	<i>Cost</i>	<i>Next Router</i>
A	0	.
B	5	-
C	2	.
D	3	.
E	6	C

Dijkstra's Algorithm

```
1 Dijkstra's Algorithm ( )
2 {
3     // Initialization
4     Tree = {root}           // Tree is made only of the root
5     for (y = 1 to N)       // N is the number of nodes
6     {
7         if (y is the root)
8             D[y] = 0        // D[y] is shortest distance from root to node y
9         else if (y is a neighbor)
10            D[y] = c[root][y] // c[x][y] is cost between nodes x and y in LSDB
11        else
12            D[y] =  $\infty$ 
13    }
14    // Calculation
15    repeat
16    {
17        find a node w, with D[w] minimum among all nodes not in the Tree
18        Tree = Tree  $\cup$  {w}      // Add w to tree
19        // Update distances for all neighbor of w
20        for (every node x, which is neighbor of w and not in the Tree)
21        {
22            D[x] = min{D[x], (D[w] + c[w][x])}
23        }
24    } until (all nodes included in the Tree)
25 } // End of Dijkstra
```



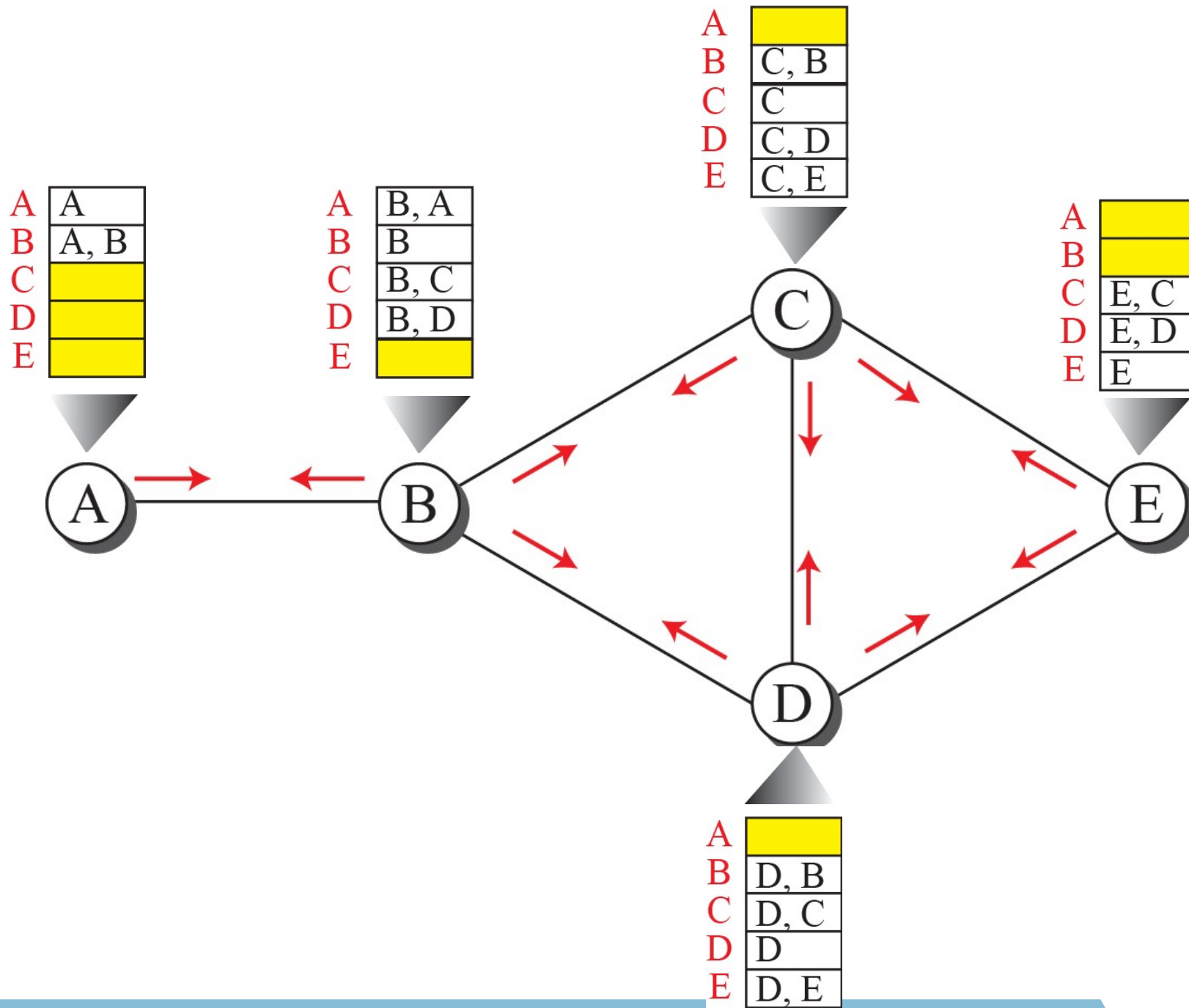
Path-Vector Routing

Both link-state and distance-vector routing are based on the least-cost goal –intra domain

Least-cost goal, applied by LS or DV routing, does not allow a sender to apply specific policies to the route a packet may take.

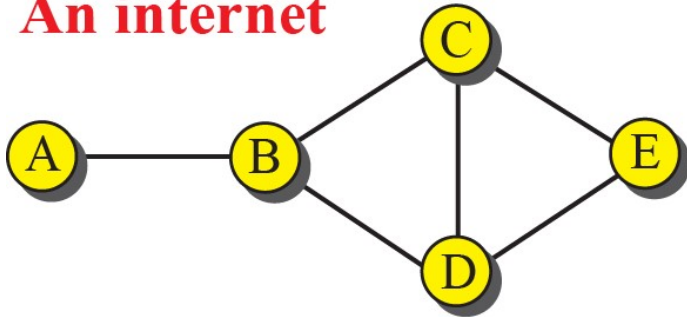
To respond to these demands, a third routing algorithm, called path-vector (PV) routing has been devised – interdomain routing

Path vectors made at booting time

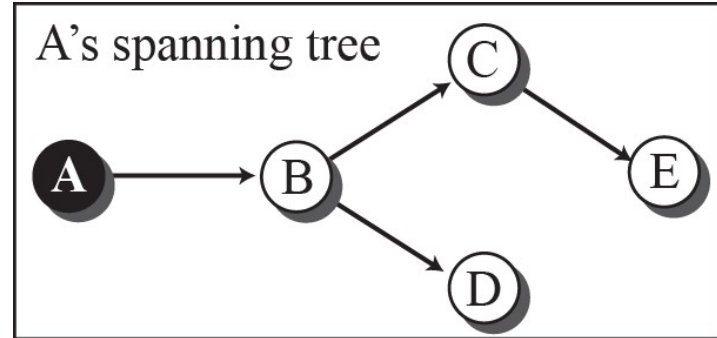


Spanning trees in path-vector routing

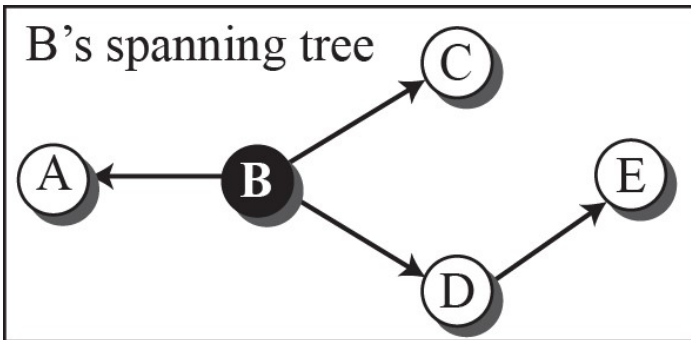
An internet



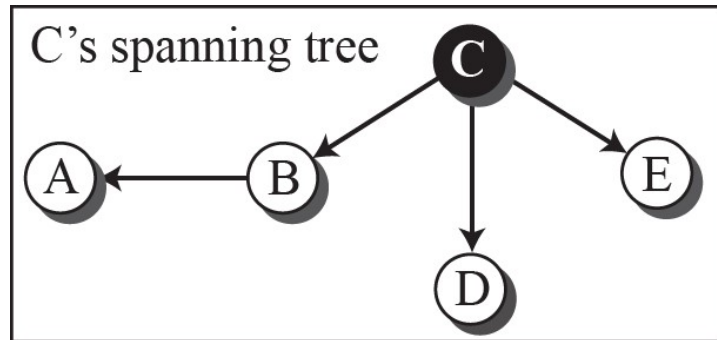
A's spanning tree



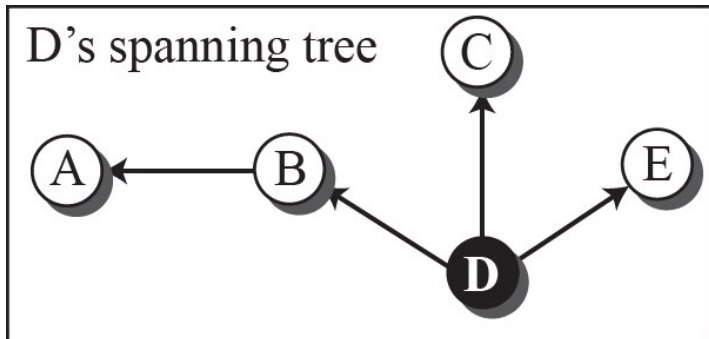
B's spanning tree



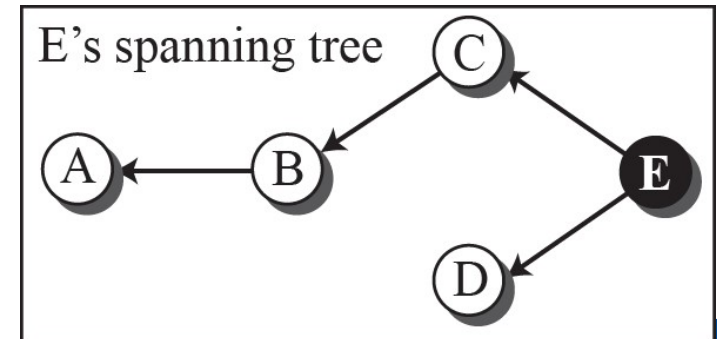
C's spanning tree



D's spanning tree



E's spanning tree



Updating path vectors

	New C		Old C		B
A	C, B, A	A		A	B, A
B	C, B	B	C, B	B	B
C	C	C	C	C	B, C
D	C, D	D	C, D	D	B, D
E	C, E	E	C, E	E	

$C[] = \text{best}(C[], C + B[])$

Note:
X []: vector X
Y: node Y

Event 1: C receives a copy of B's vector

	New C		Old C		D
A	C, B, A	A	C, B, A	A	
B	C, B	B	C, B	B	D, B
C	C	C	C	C	D, C
D	C, D	D	C, D	D	D
E	C, E	E	C, E	E	D, E

$C[] = \text{best}(C[], C + D[])$

Event 2: C receives a copy of D's vector

Path-vector algorithm for a node

```
1 Path_Vector_Routing ( )
2 {
3     // Initialization
4     for (y = 1 to N)
5     {
6         if (y is myself)
7             Path[y] = myself
8         else if (y is a neighbor)
9             Path[y] = myself + neighbor node
10        else
11            Path[y] = empty
12    }
13    Send vector { Path[1], Path[2], ..., Path[y] } to all neighbors
14    // Update
15    repeat (forever)
16    {
17        wait (for a vector Pathw from a neighbor w)
18        for (y = 1 to N)
19        {
20            if (Pathw includes myself)
21                discard the path // Avoid any loop
22            else
23                Path[y] = best { Path[y], (myself + Pathw[y]) }
24        }
25        If (there is a change in the vector)
26            Send vector { Path[1], Path[2], ..., Path[y] } to all neighbors
27    }
28 } // End of Path Vector
```

Initial routing tables in path vector routing

Dest. Path

A1	AS1
A2	AS1
A3	AS1
A4	AS1
A5	AS1

A1 Table

AS 1

Dest. Path

C1	AS3
C2	AS3
C3	AS3

C1 Table

AS 3

Dest. Path

D1	AS4
D2	AS4
D3	AS4
D4	AS4

D1 Table

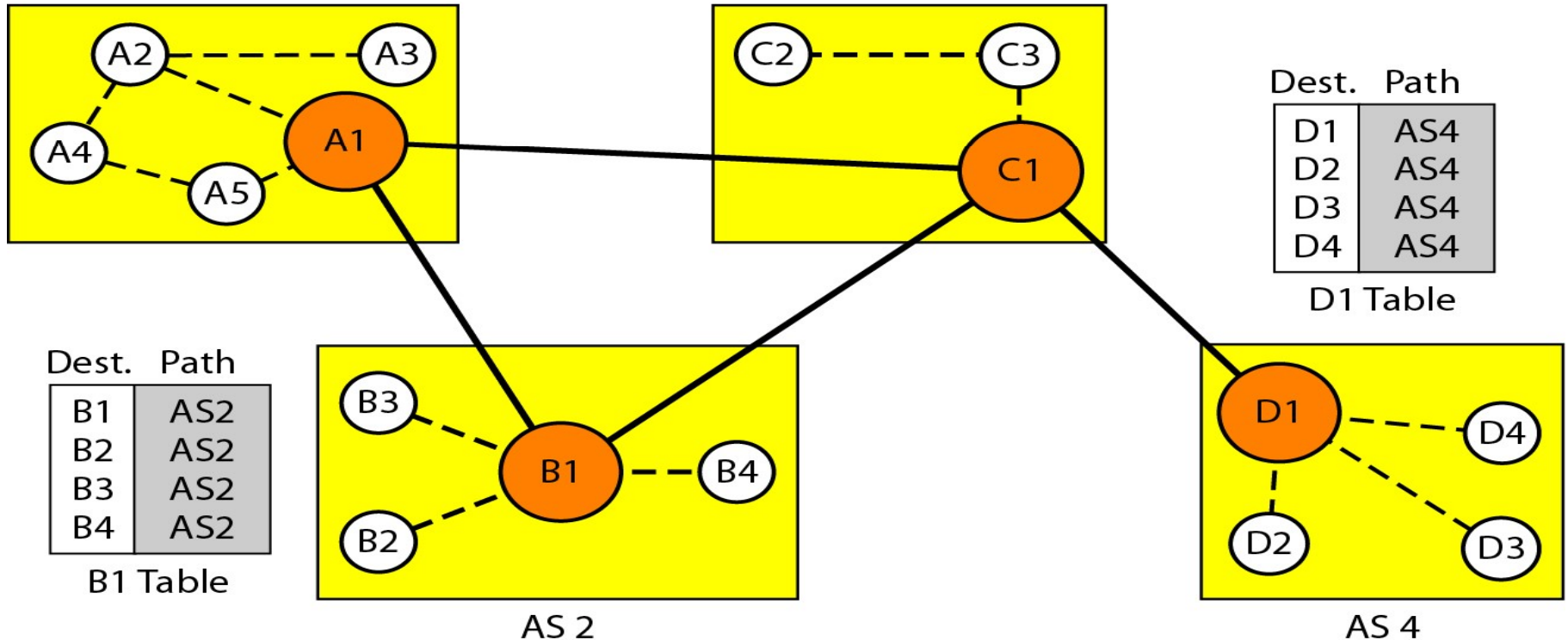
AS 4

Dest. Path

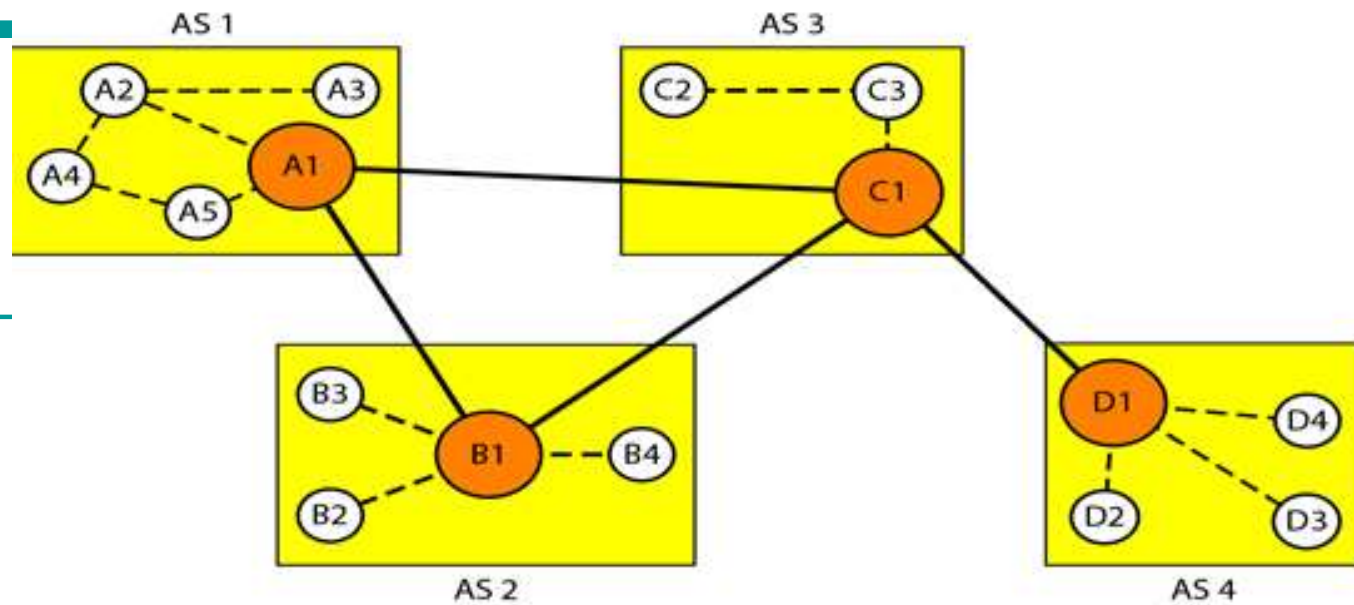
B1	AS2
B2	AS2
B3	AS2
B4	AS2

B1 Table

AS 2



*Stabilized tables for
three autonomous
systems*



Dest.	Path
A1	AS1
...	
A5	AS1
B1	AS1-AS2
...	...
B4	AS1-AS2
C1	AS1-AS3
...	...
C3	AS1-AS3
D1	AS1-AS3-AS4
...	...
D4	AS1-AS3-AS4

A1 Table

Dest.	Path
A1	AS2-AS1
...	
A5	AS2-AS1
B1	AS2
...	...
B4	AS2
C1	AS2-AS3
...	...
C3	AS2-AS3
D1	AS2-AS3-AS4
...	...
D4	AS2-AS3-AS4

B1 Table

Dest.	Path
A1	AS3-AS1
...	
A5	AS3-AS1
B1	AS3-AS2
...	...
B4	AS3-AS2
C1	AS3
...	...
C3	AS3
D1	AS3-AS4
...	...
D4	AS3-AS4

C1 Table

Dest.	Path
A1	AS4-AS3-AS1
...	
A5	AS4-AS3-AS1
B1	AS4-AS3-AS2
...	...
B4	AS4-AS3-AS2
C1	AS4-AS3
...	...
C3	AS4-AS3
D1	AS4
...	...
D4	AS4

D1 Table

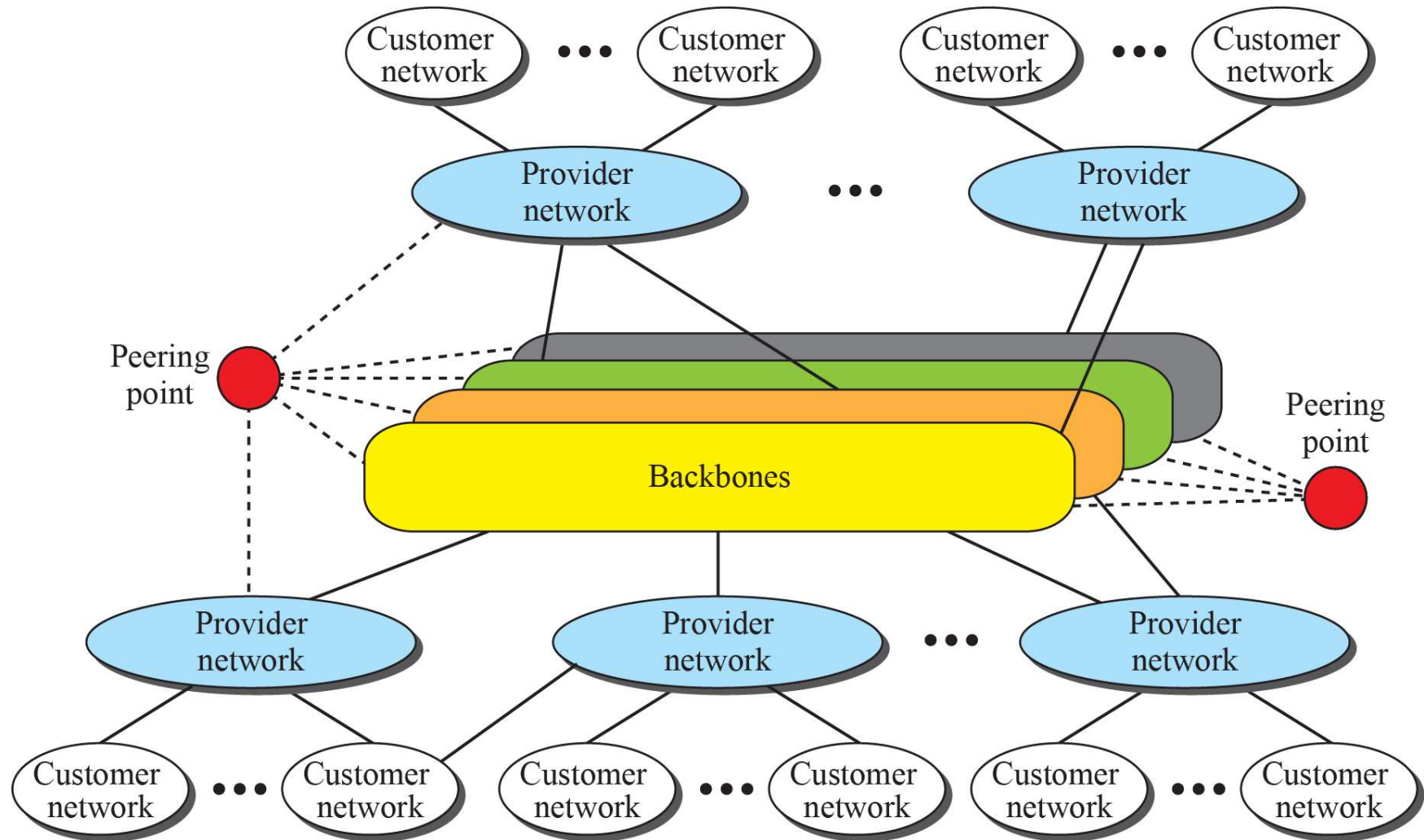
UNICAST ROUTING PROTOCOLS

Routing Information Protocol RIP - distance-vector algorithm

Open Shortest Path First OSPF -link-state algorithm

and Border Gateway Protocol BGP -path-vector algorithm

Internet structure



Hierarchical Routing

Single Protocol – not possible in internet - scalability, administrative issue

RP in AS - Intra-AS routing protocol, intradomain routing protocol, or interior gateway protocol -IGP

Global routing protocol - *inter-AS routing protocol, interdomain routing protocol, or exterior gateway protocol (EGP)*

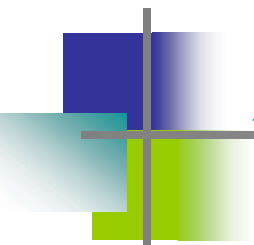
Autonomous System

Categorized – way they connect

Stub AS – connected to only one AS – data traffic can be initiated or terminated. Eg: customer nw

Multihomed AS – A multihomed AS can have more than one connection to other ASs, but it does not allow data traffic to pass through it.– don't allow traffic but provides service Eg: customer who use more than one provider

Transient AS – A transient AS is connected to more than one other AS and also allows the traffic to pass through Eg: provider and backbone nw



Routing Information Protocol

RIP was started as part of the Xerox Network System (XNS), but it was the Berkeley Software Distribution (BSD) version of UNIX that helped make the use of RIP widespread.

Uses distance vector

1. RIP routers advertise the cost of reaching different Networks (destination resides) instead of reaching other nodes in a theoretical graph
2. Cost here is hop count (max 15)

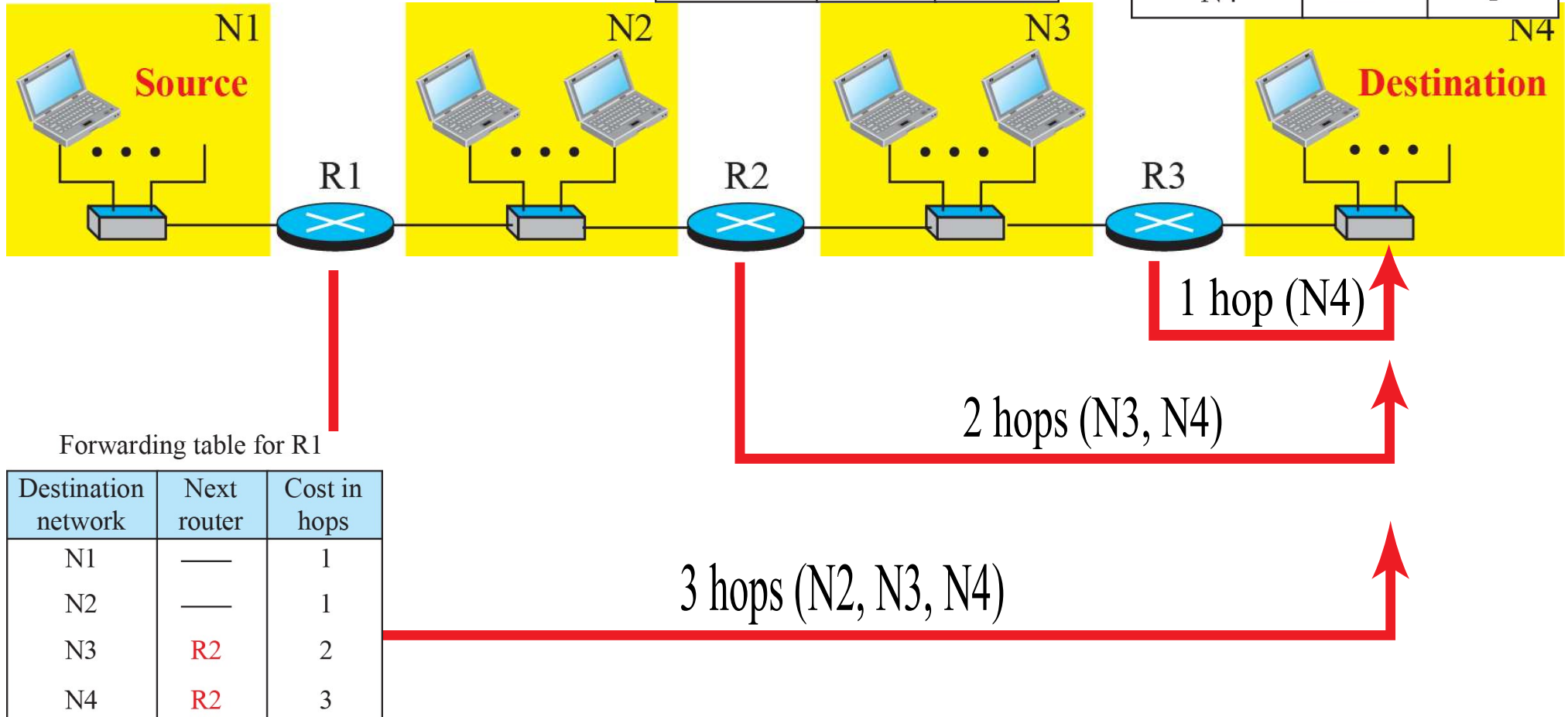
Hop counts in RIP

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	—	1
N3	—	1
N4	R3	2

Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	—	1
N4	—	1



Forwarding tables

Forwarding table for R1

Destination network	Next router	Cost in hops
N1	—	1
N2	—	1
N3	R2	2
N4	R2	3

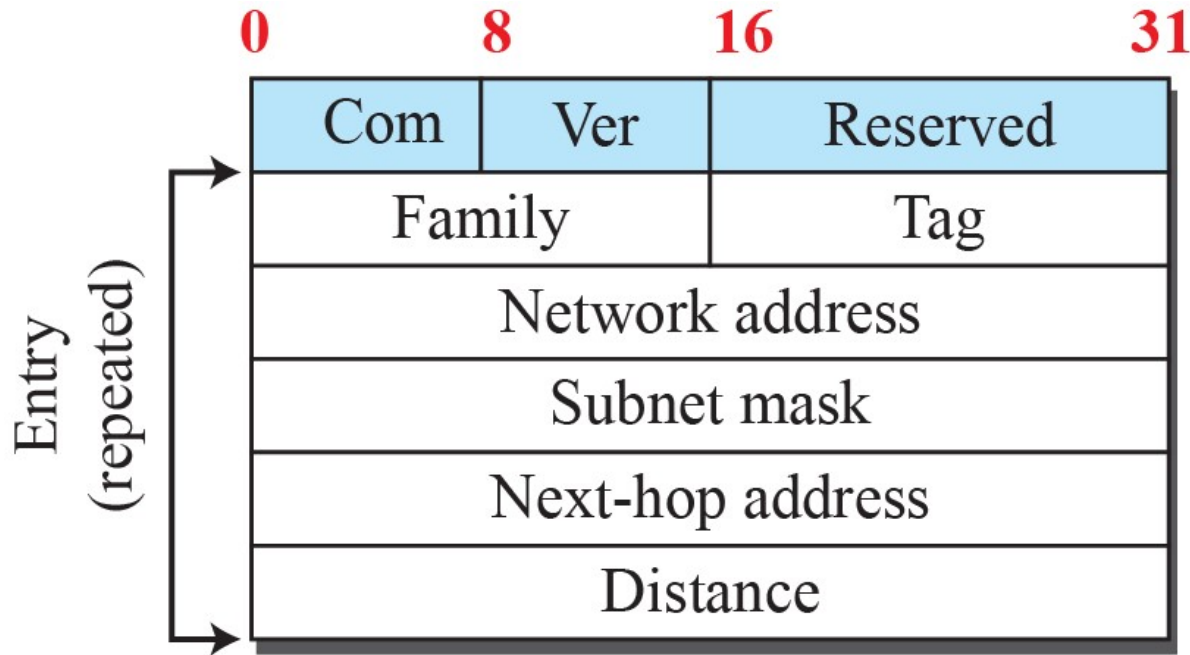
Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	—	1
N4	—	1

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	—	1
N3	—	1
N4	R3	2

RIP message format



RIP port no -
520

Fields

Com: Command, request (1), response (2)

Ver: Version, current version is 2

Family: Family of protocol, for TCP/IP value is 2

Tag: Information about autonomous system

Network address: Destination address

Subnet mask: Prefix length

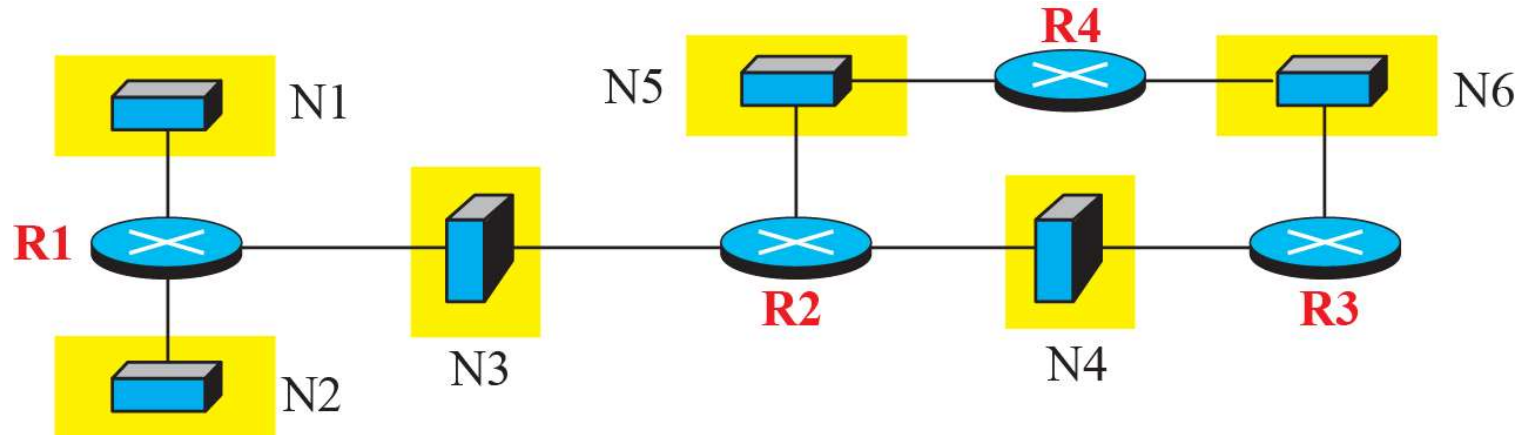
Next-hop address: Address length

Distance: Number of hops to the destination

RIP Algorithm

- Instead of sending only distance vectors, a router needs to send the whole contents of its forwarding table in a response message.
- The receiver adds one hop to each cost and changes the next router field to the address of the sending router. We call each route in the modified forwarding table the *received route* and each route in the old forwarding table the *old route*.
- The received router selects the old routes as the new ones except in the following three cases:
 1. New entry -added to the route.
 2. Less cost - selected as the new one.
 3. Cost $RR > \text{old route}$, next hop –same - selected as the new one. This is the case where the route was actually advertised by the same router in the past, but now the situation has been changed
- RIP Timers
- Periodic Timer –update
- Expiration Timer – 180 s –validity of route – receives update – no response – makes 16
- Garbage collection Timer – before purging from RT -120s

Example of an autonomous system using RIP (Part I)



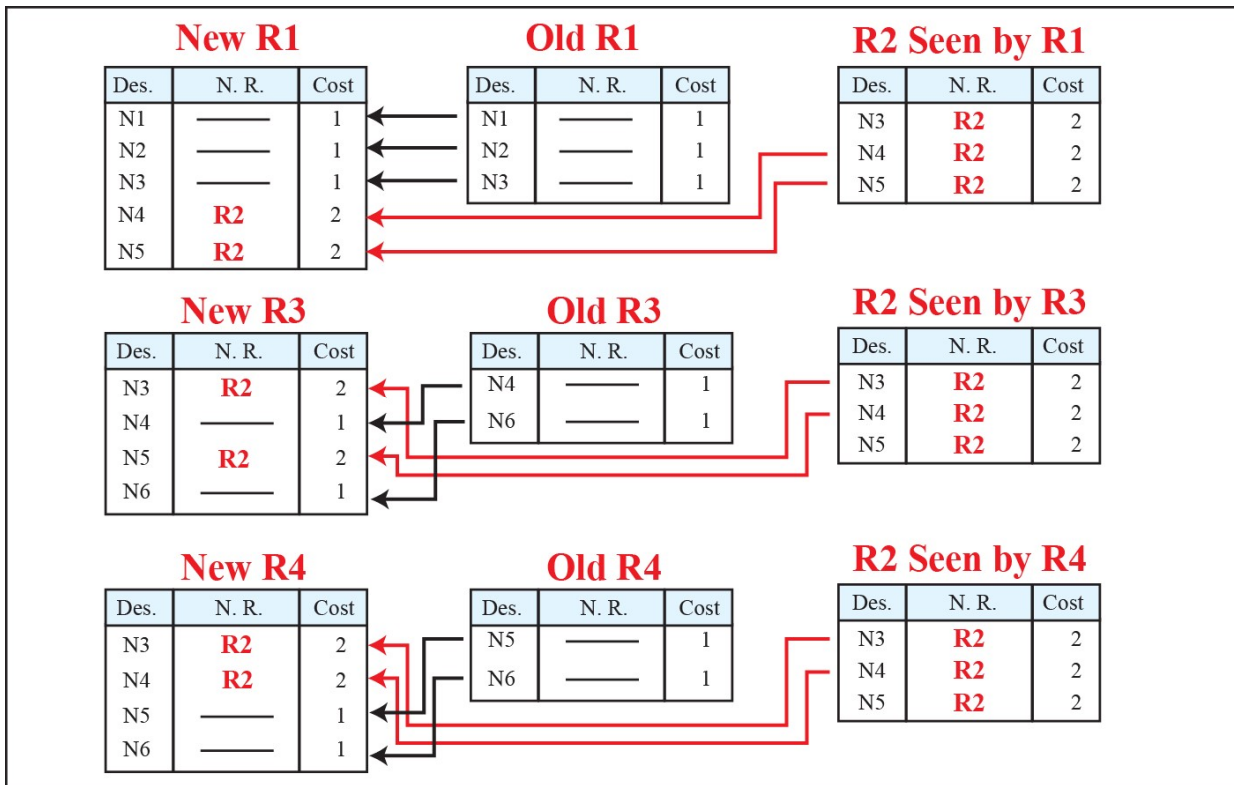
Legend

Des.: Destination network
N. R.: Next router
Cost: Cost in hops

R1			R2			R3			R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	_____	1	N3	_____	1	N4	_____	1	N5	_____	1
N2	_____	1	N4	_____	1	N6	_____	1	N6	_____	1
N3	_____	1	N5	_____	1						

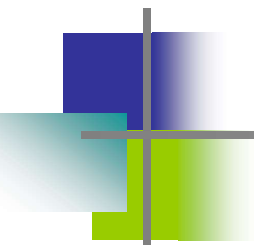
Forwarding tables after all routers booted

Example of an autonomous system using RIP (Part II)



Changes in the forwarding tables of R1, R3, and R4 after they receive a copy of R2's table

Final R1			Final R2			Final R3			Final R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1	—	1	N1	R1	2	N1	R2	3	N1	R2	3
N2	—	1	N2	R1	2	N2	R2	3	N2	R2	3
N3	—	1	N3	—	1	N3	R2	2	N3	R2	2
N4	R2	2	N4	—	1	N4	—	1	N4	R2	2
N5	R2	2	N5	—	1	N5	R2	2	N5	—	1
N6	R2	3	N6	R3	2	N6	—	1	N6	—	1

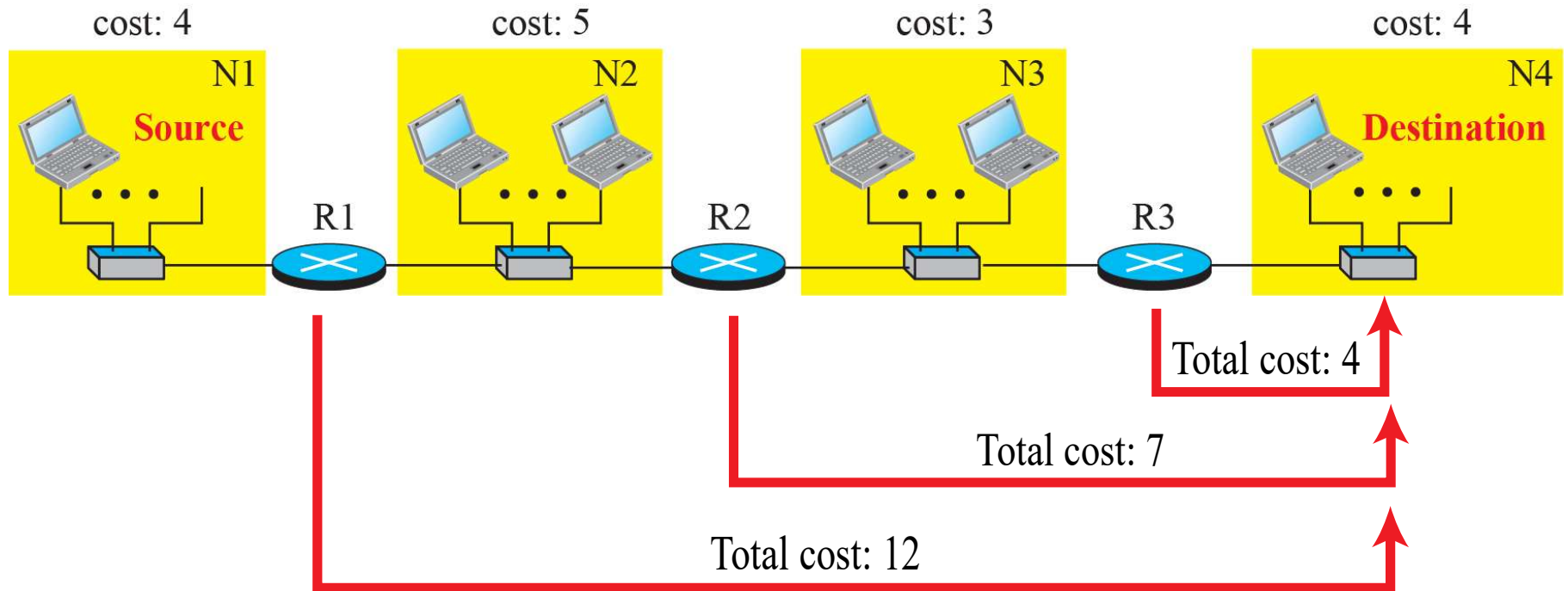


Open Shortest Path First

Open Shortest Path First (OSPF) is also an intradomain routing protocol

Cost – different type of service such as RTT, throughput– weights will vary

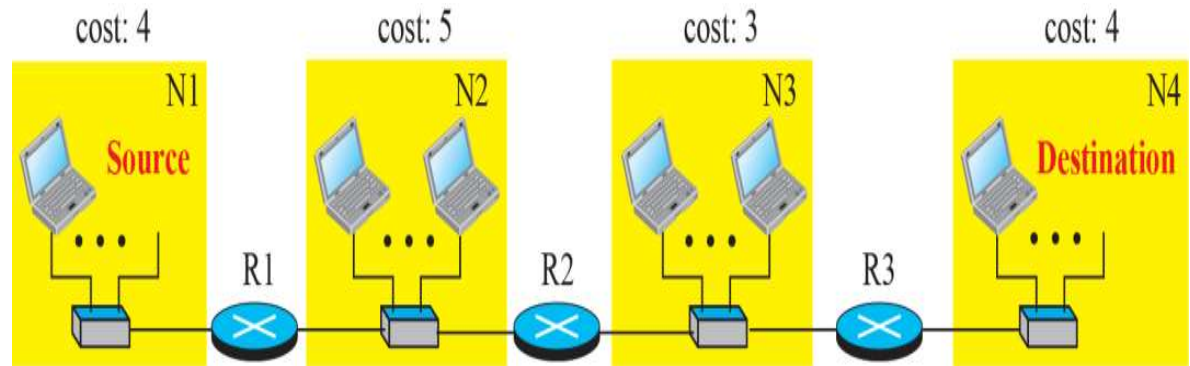
Metric in OSPF



Forwarding tables in OSPF

Forwarding table for R1

Destination network	Next router	Cost
N1	—	
N2	—	
N3	R2	
N4	R2	12



The internet from previous figure

Forwarding table for R3

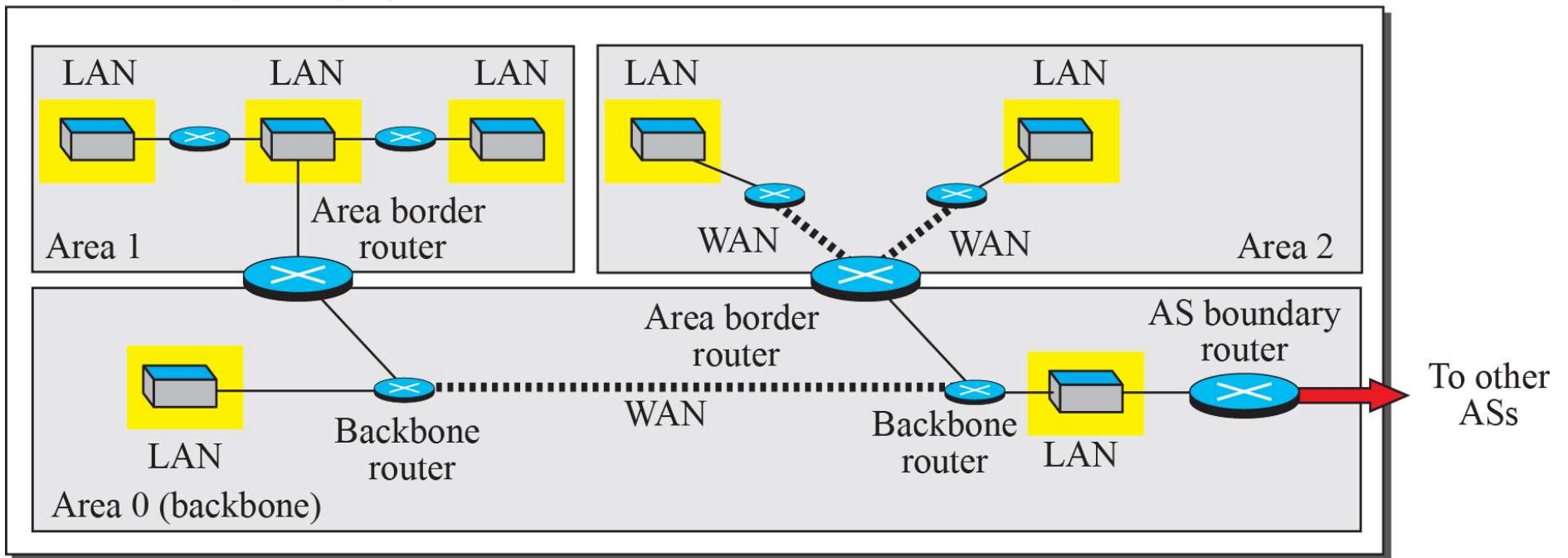
Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	—	3
N4	—	4

Forwarding table for R2

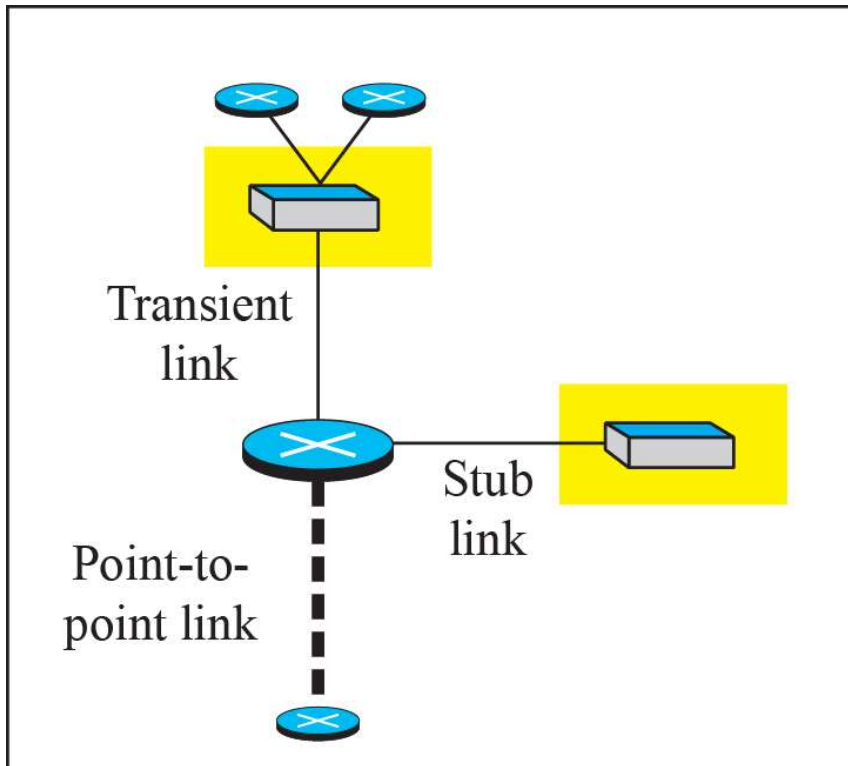
Destination network	Next router	Cost
N1	R1	9
N2	—	5
N3	—	3
N4	R3	7

Areas in an autonomous system

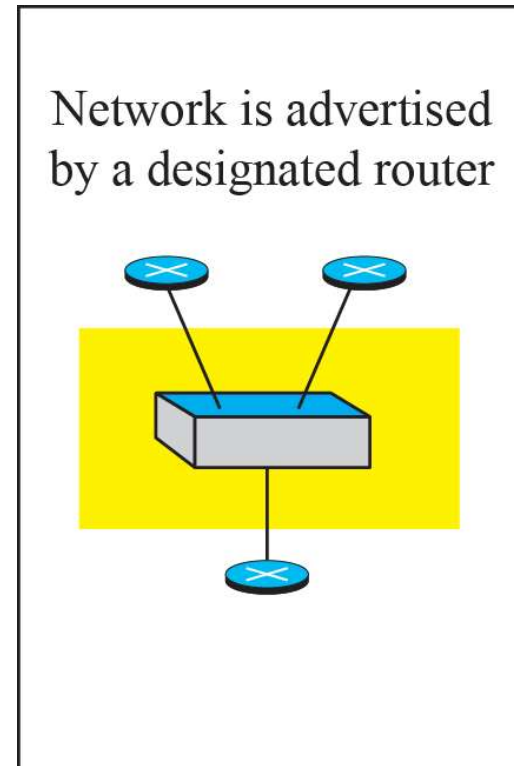
Autonomous System (AS)



Five different LSPs (Part I)

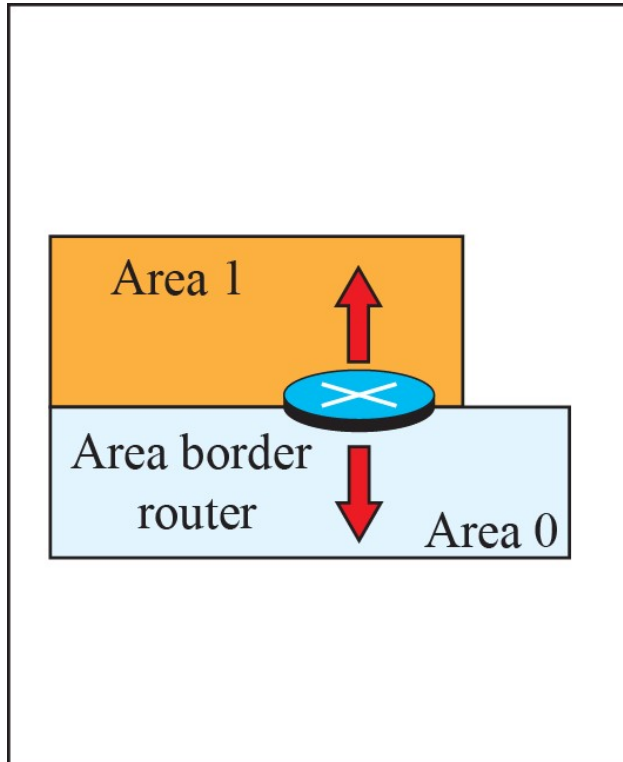


a. Router link

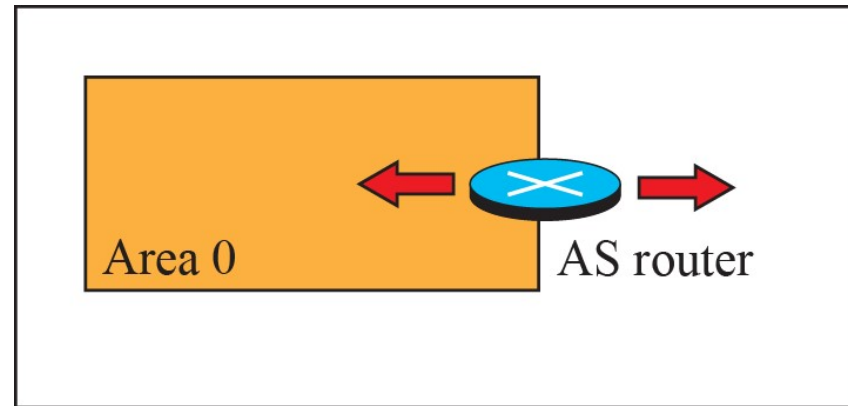


b. Network link

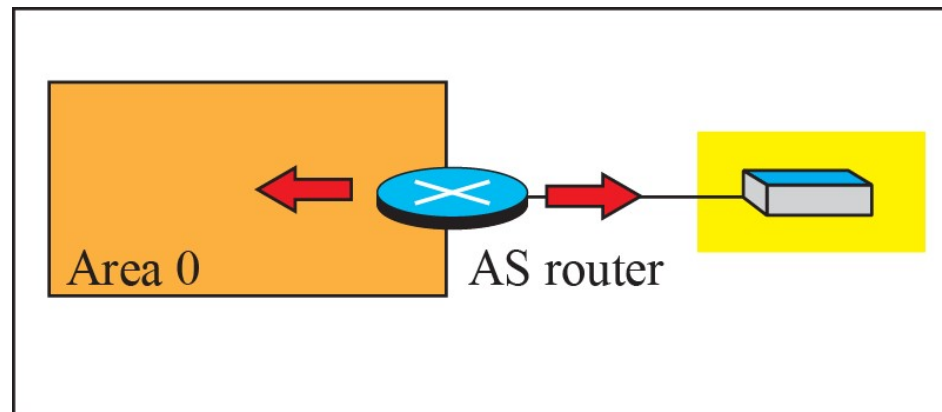
Five different LSPs (Part II)



c. Summary link to network



d. Summary link to AS



e. External link

OSPF message formats (Part I)

0	8	16	31
Version	Type	Message length	
Source router IP address			
Area Identification			
Checksum		Authentication type	
Authentication			

OSPF common header

Legend

E, T, B, I, M, MS: flags used by OSPF
 Priority: used to define the designated router
 Rep.: Repeated as required

OSPF common header (Type: 1)			
Network mask			
Hello interval		E	T
Dead interval			
Designated router IP address			
Backup designated router IP address			
Neighbor IP address			

Rep. [

Attention →

OSPF common header (Type: 2)			
		E	B
		I	M
		MS	
Message sequence number			
Link-state general header			

Database description

Type 1 –Hello by router

Type 2- DB description as response – LSDB

Type 3 – LS Request

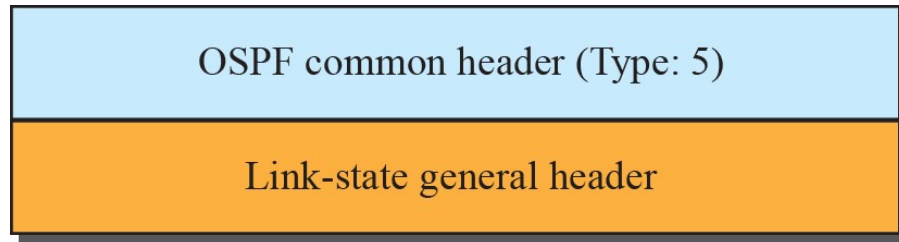
Type 4 – LS update – 5 diff links

Type 5 – LS Ack

OSPF message formats (Part II)

LS age		E	T	LS type
LS ID				
Advertising router				
LS sequence number				
LS checksum		Length		

Link-state general header

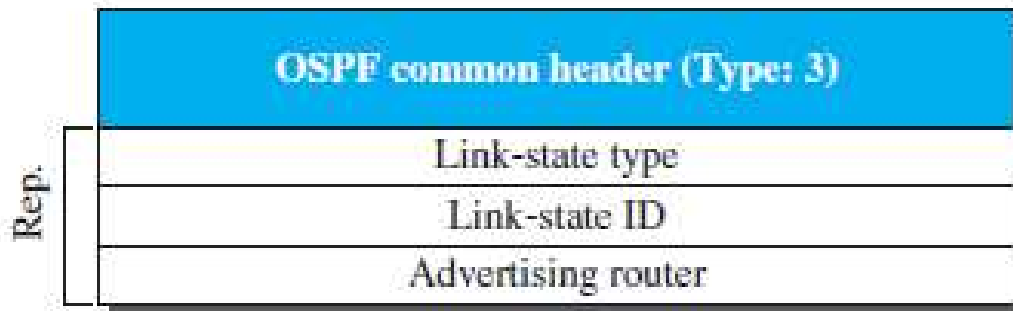
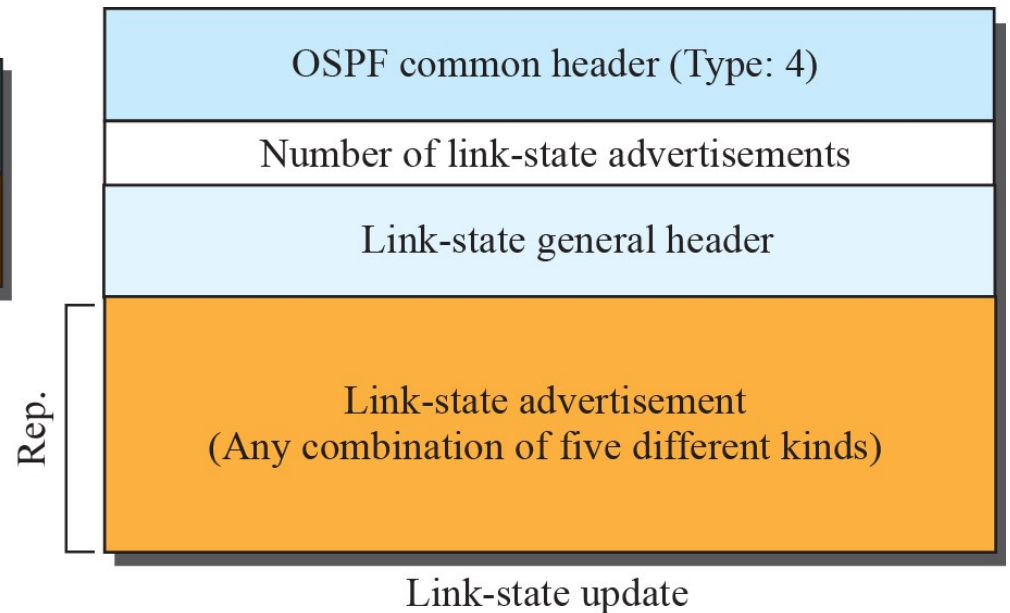


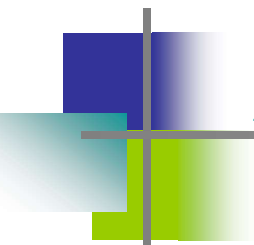
Link-state acknowledgment

Attention →

Legend

E, T, B, I, M, MS: flags used by OSPF
 Priority: used to define the designated router
 Rep.: Repeated as required



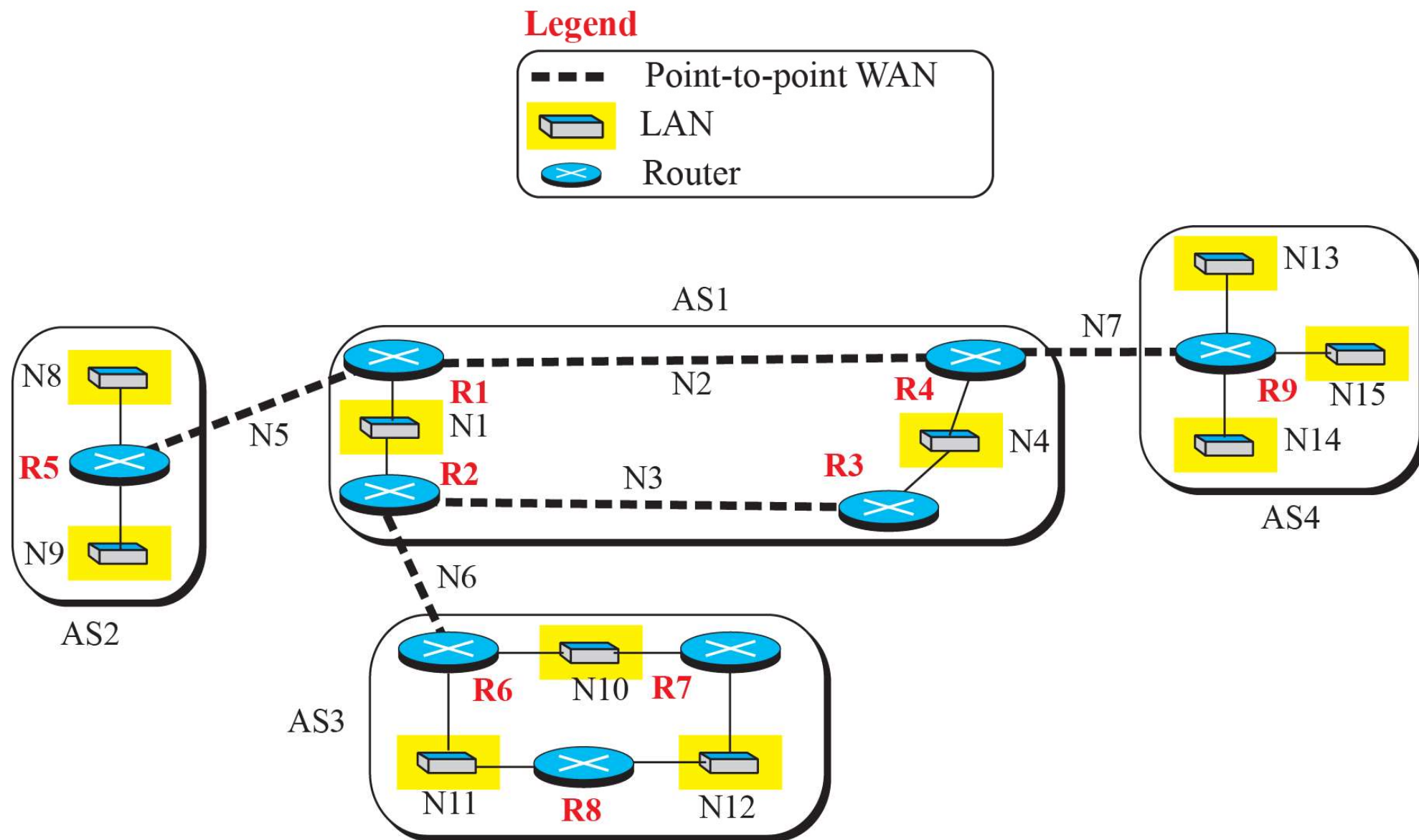


Border Gateway Protocol

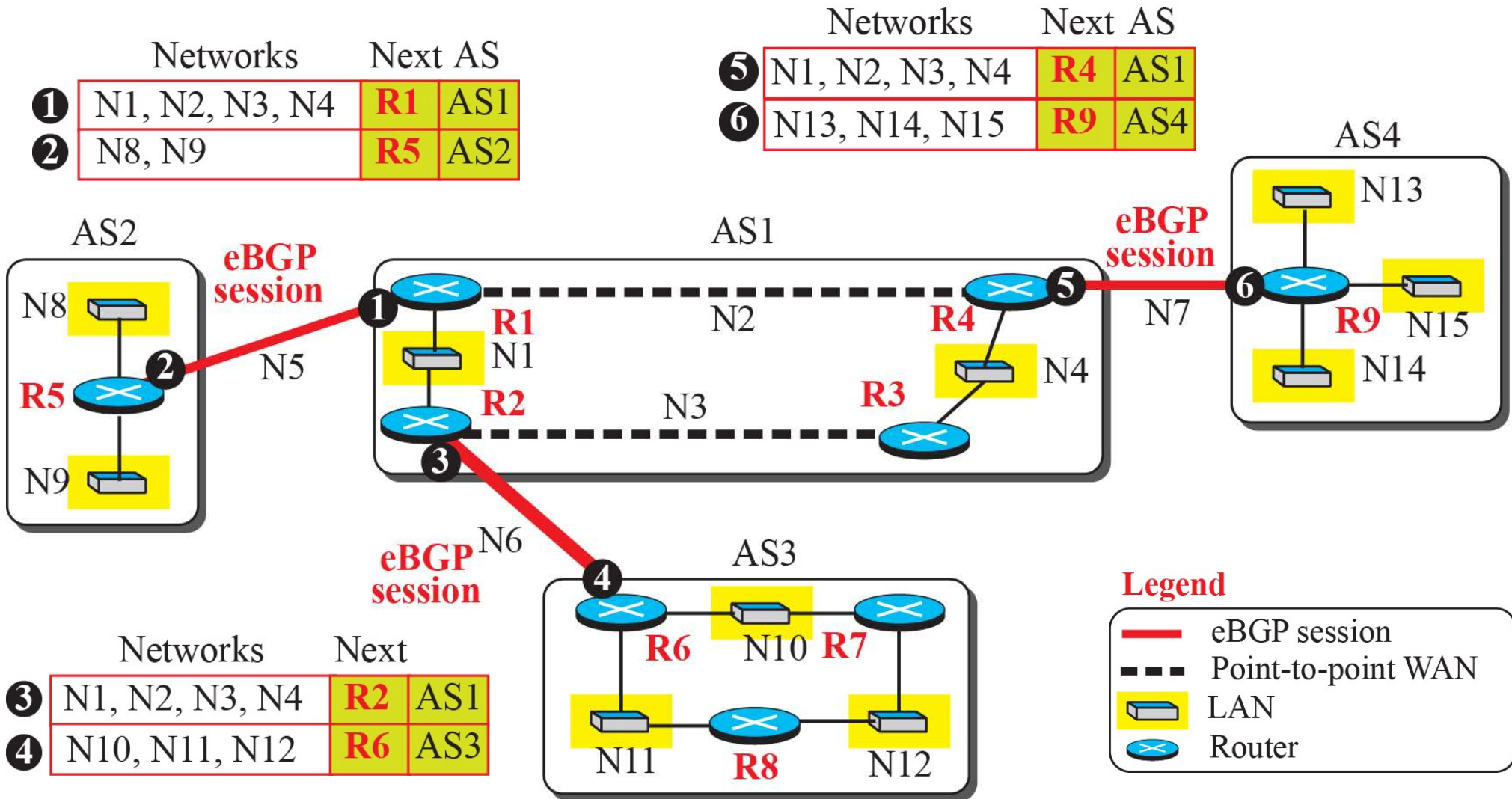
The Border Gateway Protocol version 4 (BGP4) is the only interdomain routing protocol used in the Internet today.

BGP4 is based on the path-vector algorithm - provide information about the reachability of networks in the Internet.

A sample internet with four ASs



eBGP operation



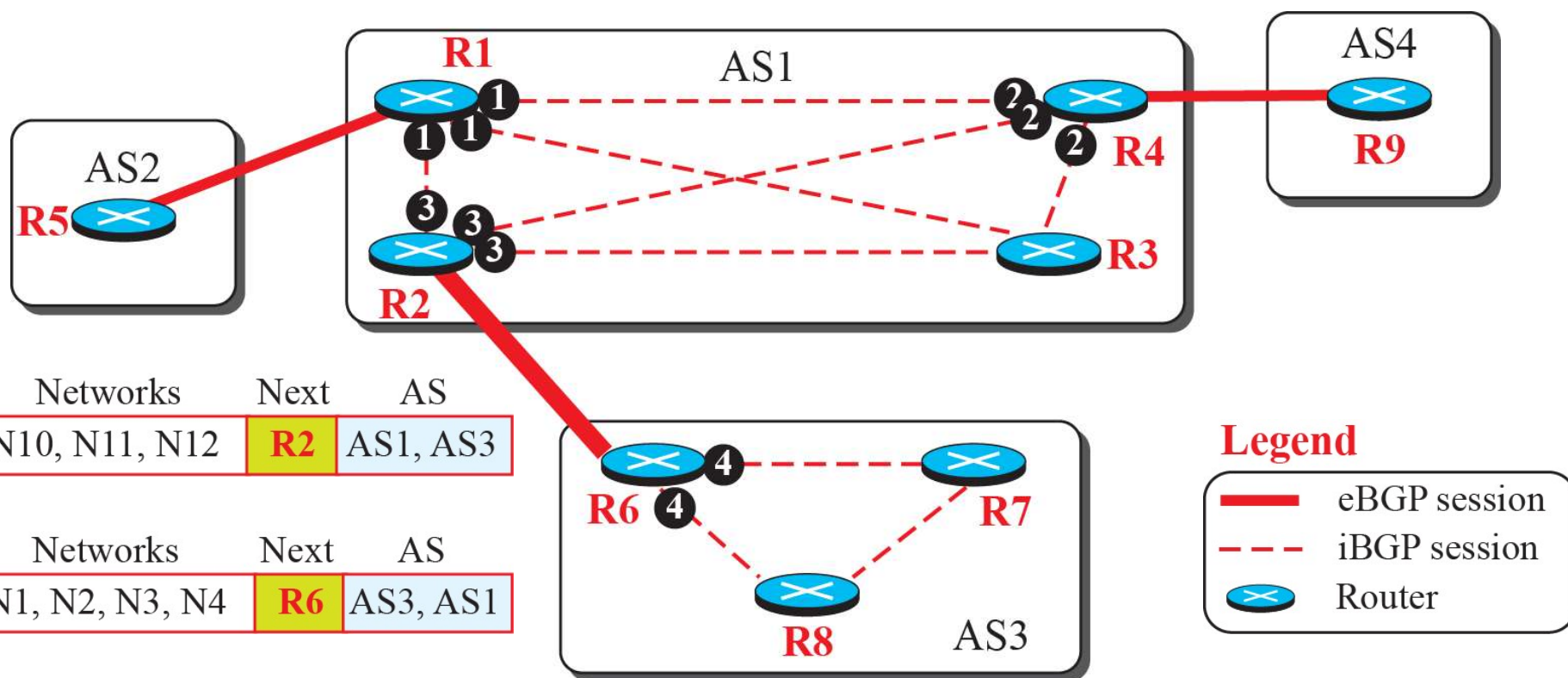
Combination of eBGP and iBGP sessions in our internet

	Networks	Next	AS
1	N8, N9	R1	AS1, AS2

	Networks	Next	AS
2	N13, N14, N15	R4	AS1, AS4

	Networks	Next	AS
3	N10, N11, N12	R2	AS1, AS3

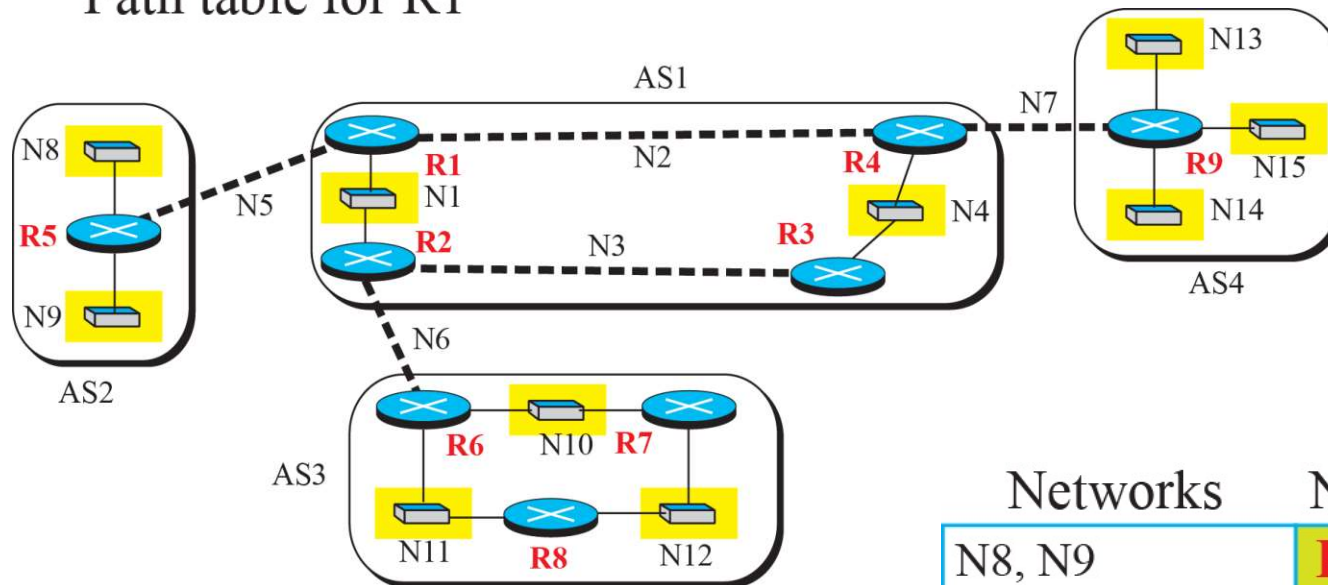
	Networks	Next	AS
4	N1, N2, N3, N4	R6	AS3, AS1



Finalized BGP path tables (Part I)

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R1



Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R3

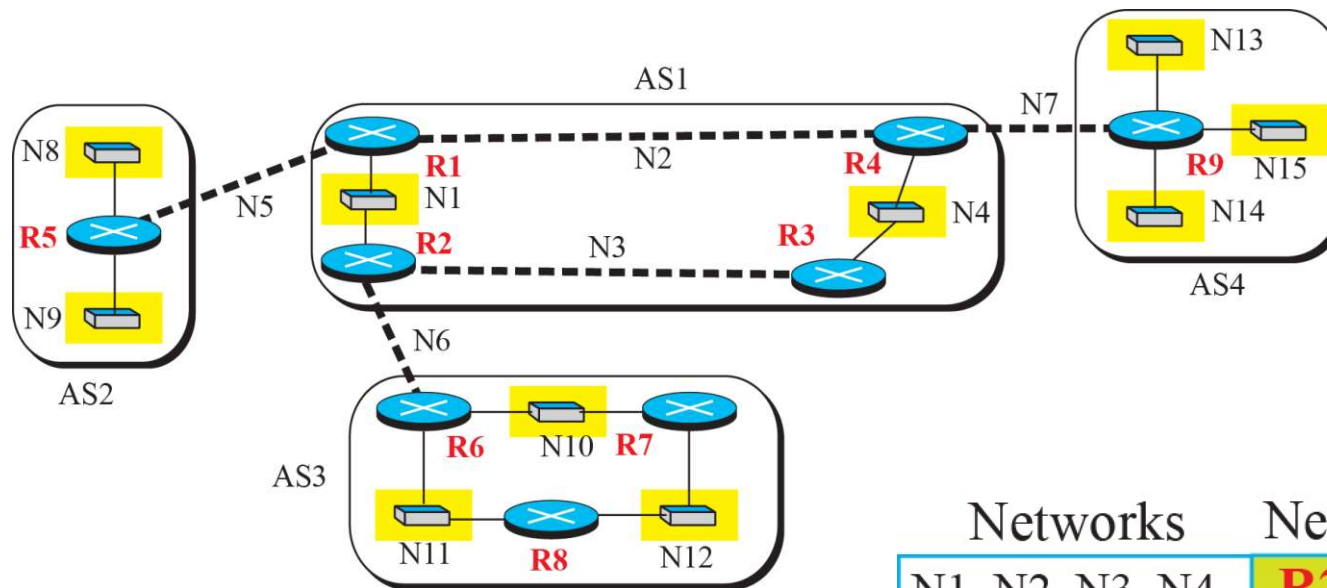
Finalized BGP path tables (Part II)

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R5

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R4



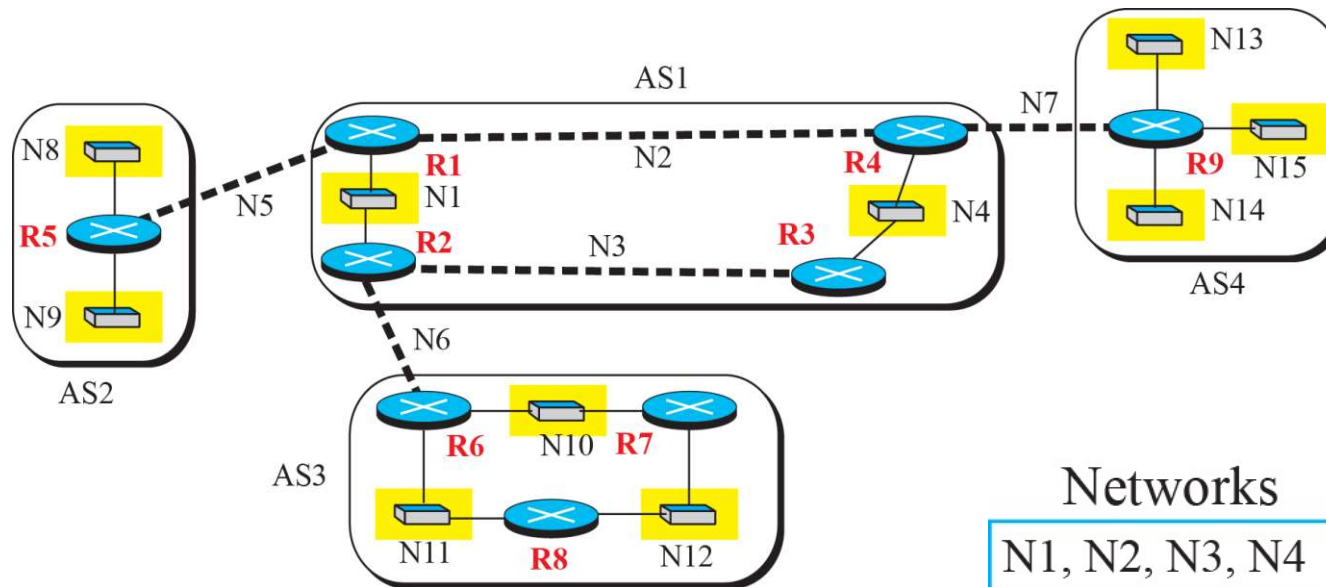
Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R6

Finalized BGP path tables (Part III)

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Path table for R9



Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Forwarding tables after injection from BGP (Part I)

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

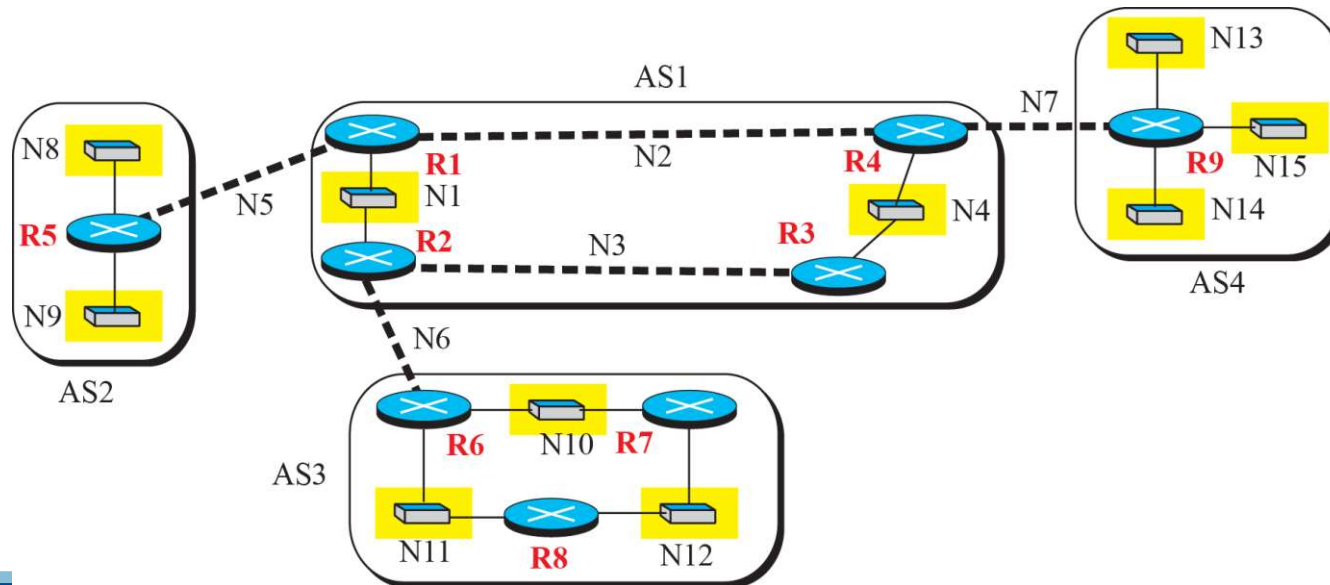
Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4



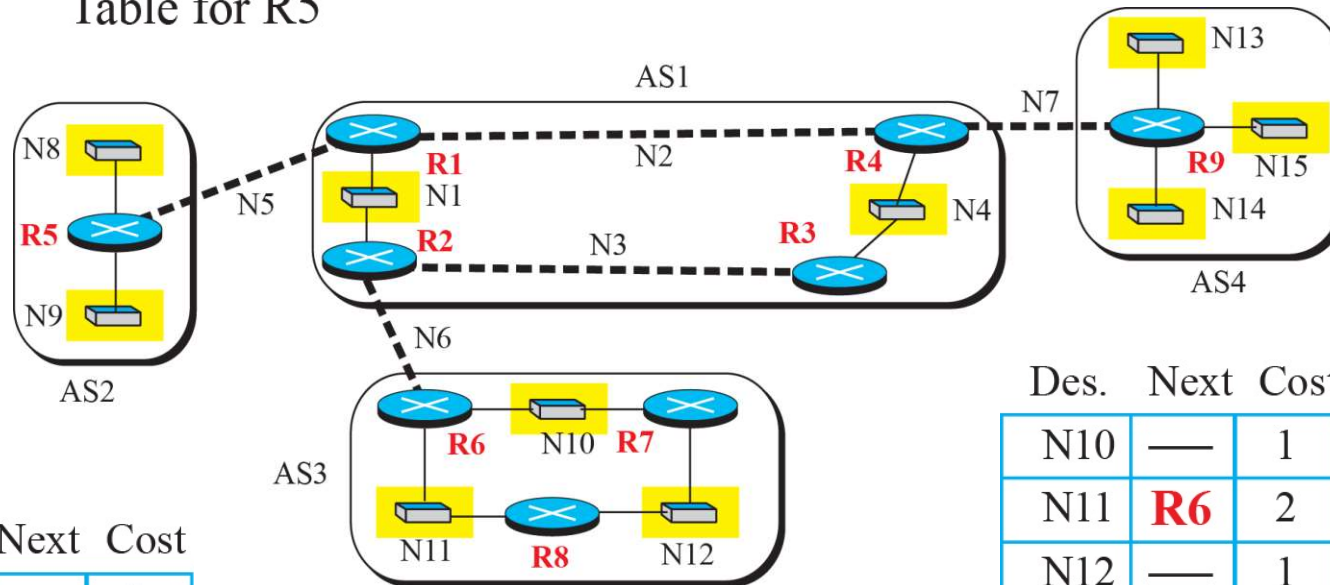
Forwarding tables after injection from BGP (Part II)

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

Table for R9



Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

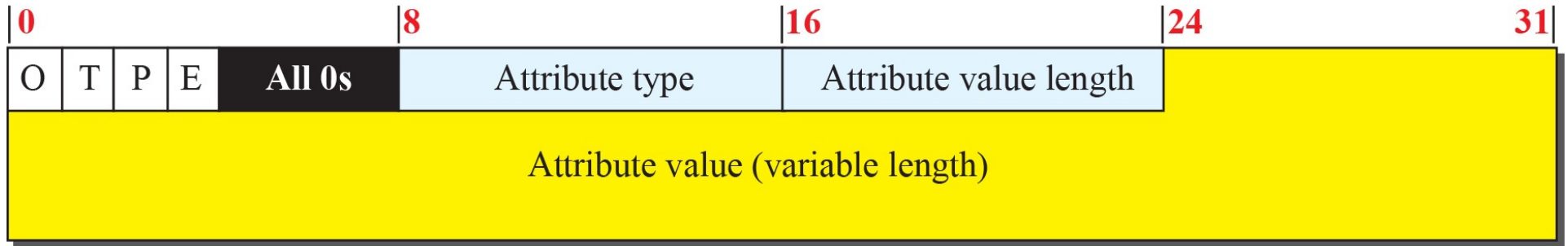
Table for R7

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Format of path attribute

- O:** Optional bit (set if attribute is optional)
P: Partial bit (set if an optional attribute is lost in transit)
T: Transitive bit (set if attribute is transitive)
E: Extended bit (set if attribute length is two bytes)

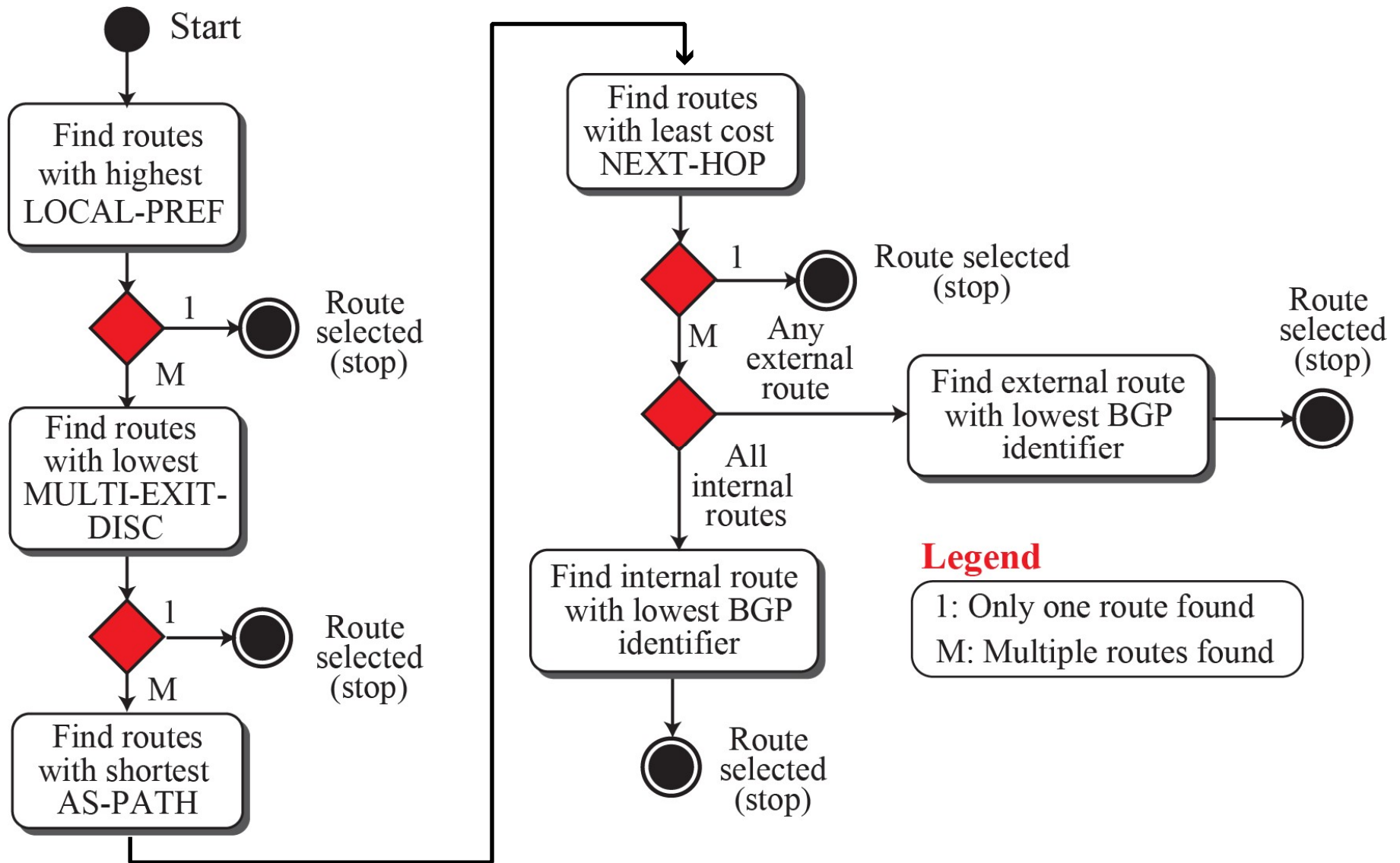


Type

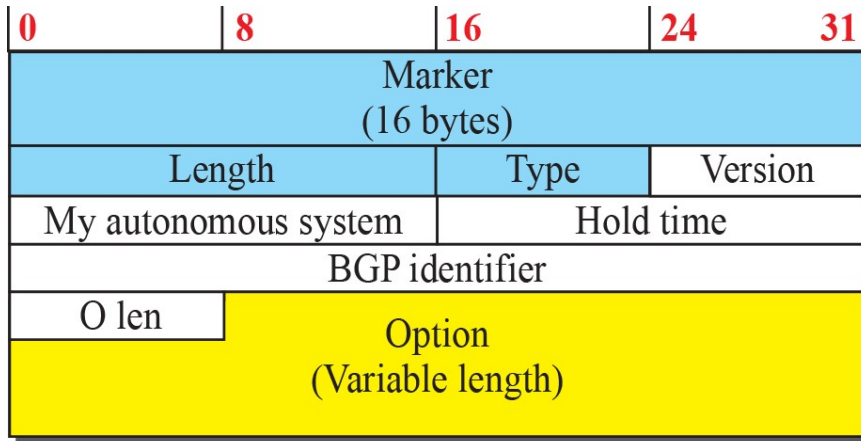
- 1: ORIGIN
- 2: AS_PATH
- 3: NEXT_HOP
- 4: MULTI_EXIT_DISC
- 5: LOCAL_PREF
- 6: ATOMIC_AGGREGATE
- 7: AGGREGATOR



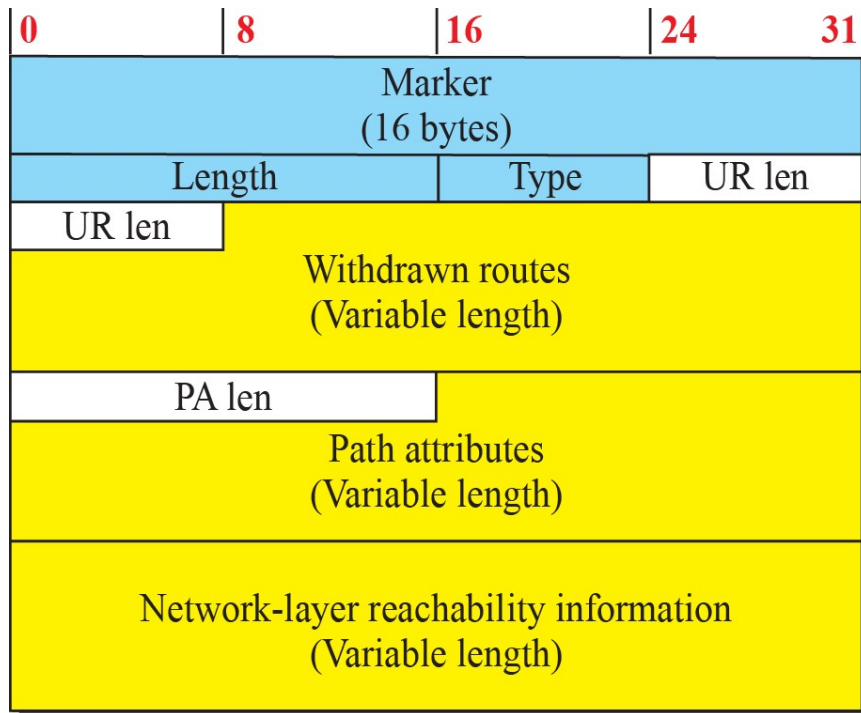
Flow diagram for route selection



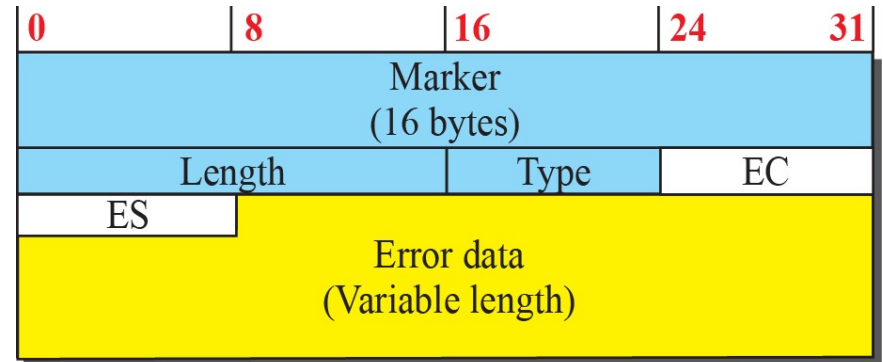
BGP messages



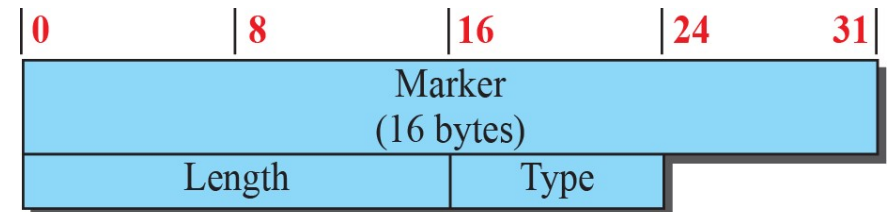
Open message (type 1)



Update message (type 2)



Notification message (type 3)



Keepalive message (type 4)

Fields in common header

Marker: Reserved for authentication

Length: Length of total message in bytes

Type: Type of message (1 to 4)

Abbreviations

O len: Option length

EC: Error code

ES: Error subcode

UR len: Unfeasible route length

PA len: Path attribute length

Check Your Understanding

- Difference between RIP and OSPF
- How does a router get information about other AS?
- If there are n routers, then how many iBGP sessions are needed?

Outcome

Student should be able to understand

- Routing Algorithms
 - Distance Vector
 - Link state Routing
 - Path Vector Routing
- Routing Protocols
 - Routing Information Protocol RIP
 - Open Shortest Path First OSPF
 - Border Gateway Protocol BGP

THANK YOU